

Institutional Books — Enriched Text

A customizable multilingual open-source pipeline for denoising,
deduplicating, and annotating OCR text at scale

David Lowry-Duda¹, Matteo Cargnelutti¹, Catherine Brobston¹, Salwa Ismail²,
Greg Leppert^{†1}, Amanda Watson³, and Jonathan Zittrain⁴

¹Institutional Data Initiative, Harvard Law School Library

²Harvard Library

³Harvard Law School Library

⁴Harvard Law School, Harvard School of Engineering and Applied Sciences, Harvard Kennedy School

Abstract

Released in 2025, Institutional Books: Harvard Library (IB-HL) is a collection of 983,004 volumes (242B o200k_BASE tokens), originally digitized through Harvard Library’s participation in the Google Books Library project. As researchers and developers have begun to use IB-HL, a tension has emerged between standard large-scale preprocessing practices and the goals of careful information stewardship. Many existing pipelines optimize for web text: as a result, they tend to aggressively filter, deduplicate, restrict by language, and sometimes discard meaningful metadata. Meanwhile, researchers seeking to use IB-HL duplicate effort while performing similar processing and analysis.

We describe an approach that we call **Enriched Text**. Instead of producing a single “complete” stream of tokens, we normalize the text while preserving metadata through annotations. We separate end-matter, detect per-paragraph language, identify clusters of duplicate paragraphs, and compute per-paragraph bits-per-byte scores. We provide this information through HTML-like annotations layered on top of the text. By parsing these annotations, users can tailor the output to their own needs instead of accepting a global editorial decision on content. The pipeline applies to all ≈ 250 languages in the collection.

This report describes this project’s goals, implementation, and design rationale. The release includes IB-HL-ET (an enriched-text version of IB-HL containing 217B o200k_BASE tokens across 983,003 volumes, organized into 1.39B annotated subtopic paragraphs) and the pipeline that produced it. These serve to make the collection easier for machines to parse and for humans to study.

[†]Corresponding author gleppert@law.harvard.edu

Contents

1	Introduction	3
2	Contributions	4
3	Goals and Scope	4
4	Processing Pipeline	6
4.1	Preprocessing	6
4.2	Unicode Normalization	8
4.3	Duplicate Page Removal	9
4.4	Endmatter Separation	9
4.5	Dehyphenation	11
4.6	Running Header and Footer Removal	13
4.7	Page Number Removal	15
4.8	Sentence Segmentation	16
4.9	Topical Chunking	18
4.10	Duplicate Identification	19
4.11	Bits-Per-Byte	24
4.12	Final Annotation and Formatting	26
5	Dataset Analysis	30
5.1	Tokenizability in IB-HL and IB-HL-ET	30
5.2	Multilingual Reuse of Duplicated Text	31
5.3	Bits-Per-Byte Over Time	32
6	Discussion and Further Directions	33
7	Acknowledgements	34
8	Rights Determination	35
9	Disclaimers	35
	Reference list	36
	Appendix A Dataset Fields	44
	Appendix B Additional related work	44
	Appendix C Details on Processing	45
C.1	Preprocessing Details	45
C.2	Unicode Normalization	46
C.3	Duplicate Page Removal	47
C.4	Endmatter Separation	48
C.5	Dehyphenation	48
C.6	Running Headers and Footers	50
C.7	Page Number Removal	50
C.8	Segmentation Details	50
C.9	Alternate Chunking Algorithms Considered	53
C.10	Duplicate Identification Details	53
C.11	Bits-Per-Byte Details	54
	Appendix D Examples with Known Limitations	56
	Appendix E Using the Parser Library	57

1 Introduction

In 2025, the Institutional Data Initiative introduced Institutional Books: Harvard Library (IB-HL)¹ (Cargnelutti et al. 2025), a 242B-token dataset of digitized books and other bound materials drawn from Harvard Library’s collections. That work emphasized rigorous information stewardship, including detailed provenance, collection-level deduplication, optical character recognition (OCR) artifact analysis, post-processing to improve legibility, and rich metadata.

Following the release of IB-HL, researchers and developers sought to use the corpus for large language model (LLM) training and processing. TYPEWRITERLM² (Luo et al. 2026) uses IB-HL as its primary source (97.7% of tokens) for its 54B-token historical pretraining corpus, for a 7.2B-parameter language model trained on pre-1913 English. Similarly, the TALKIE-1930³ (Levine, Duvenaud, and Radford 2026) 13B-parameter “vintage language model” uses IB-HL data and metadata in its 260B-token pre-1931 English corpus. The K2-V2 open-weight reasoning model (Liu et al. 2025) incorporates IB-HL as a source of long-context documents in its mid-training data.

Feedback from early adopters indicates tensions between common large-scale preprocessing practices and the goals that motivated IB-HL. LLMs depend on the nature, quality, and diversity of the data used in training, yet standard internet-oriented pipelines often rely on aggressive language filtering (such as only using English) and deduplication (such as discarding all near-duplicate text); see for example (J. Kaplan et al. 2020; Gunasekar et al. 2023; Muennighoff et al. 2023). These practices strip multilingual content and reduce document fidelity. For IB-HL, excluding non-English would discard half the corpus and narrow its cultural scope. Naive chunk-level deduplication fragments books and long-form documents. Furthermore, pipelines often flatten or discard *paratext* (Genette and Maclean 1991): titles, tables of contents, footnotes, indices, publisher notices, and other elements that mediate between a text and its readers.

We describe an alternative approach that we refer to as **Enriched Text**. Rather than producing a single “clean” stream of tokens, we offer a configurable dataset. Concretely, we represent many paratextual elements and metadata (e.g. front and back matter, detected languages, duplicate paragraph information) as machine-readable annotations layered on top of the OCRred text. We also try to identify and separate the *endmatter* (consisting of the front and back matter framing the main text, including the title page, copyright page, dedication, table of contents, foreword, glossary, bibliography, index, author’s notes) from the *middlematter* (i.e. the main text itself). These annotations are made available as attributes in an HTML-like markup language that can be quickly and efficiently parsed with standard parsing libraries, ultimately enabling users to tailor the representation to their needs.

This work focuses on three foundational problems in large-scale dataset construction:

1. **Cleaning** OCRred text across hundreds of languages and scripts while preserving underlying structure;
2. **Segmenting** volumes into semantically meaningful chunks that respect document and section boundaries; and

¹<https://huggingface.co/datasets/institutional/institutional-books-1.0>

²<https://huggingface.co/typewriter-ai/typewriter-1913-7B-base>

³<https://huggingface.co/talkie-lm/talkie-1930-13b-base>

3. Identifying Duplicate Segments across the whole collection.

By releasing IB-HL-ET, we continue IDI’s broader project of establishing a community-led, knowledge-institution-anchored approach to data. We offer a variant of IB-HL that integrates into modern information-access workflows, training pipelines, and other forms of computational access and scholarship.

2 Contributions

We introduce the following contributions:

1. A detailed technical report on the processing work we have conducted to create IB-HL-ET, an **Enriched Text** version of the Institutional Books — Harvard Library dataset sourced from Harvard Library’s collection.
2. A public dataset containing 217B o200K_BASE tokens across 7B sentences, 1.39B subtopic paragraphs, 297M subtopic sections, and 983k volumes. Available at <https://huggingface.co/datasets/institutional/institutional-books-hl-enriched-text>.
3. The Python pipeline we created to produce this collection. Available at <https://github.com/institutional/institutional-books-enriched-text-pipeline>.
4. A small, dependency-free Python library for parsing and adapting the dataset to specific use cases. Available at <https://github.com/institutional/institutional-books-enriched-text-parser>.
5. Synthetically generated training data for an endmatter classifier and subclassifier. Available at <https://github.com/institutional/institutional-books-enriched-text-pipeline> as a GitHub release.

3 Goals and Scope

IB-HL-ET is another step in a multifaceted, collaborative research process. Our goal is to make the underlying collection easier to access *computationally*, i.e. easier to read, filter, search, and study using computational methods, including AI. We treat this as a text-to-text problem, extending previous OCR and OCR-refining work. We organize and analyze existing text, but we do not re-OCR any volume. This allows the pipeline to be applied downstream of, and efficiently augment, more computationally expensive processes the outputs of which organizations may not want to discard or recreate. This may be especially true where organizations inherit existing OCR through vendor partnerships (e.g. Google Books) or their own digital archives.

Historical Collection. This project extends IB-HL and the source material from the Harvard Library collection. The same disclaimers from IB-HL apply; we advise readers to consult the disclaimer in §9.

Configurability. To improve computational access, we separate the linguistic content of a volume (its words and punctuation) from its *paratext* (reference materials that surround and mediate the main text). We prefer to label this material rather than delete it. In particular, we identify and separate endmatter from middlematter so that users can distinguish the *text* from its *context*, but still draw on that context when useful. More generally, we annotate the text so that users can *choose* how to interact with the collection. A reader who wants long-context English prose can strip all endmatter and filter only to English text using only annotations, while readers with different use cases can retain the layers and data they need.

In keeping with the stewardship goals of IB-HL, we prioritize fidelity to the source material and prefer to *mark* and *annotate* rather than rewrite or remove. However, in building a highly configurable dataset and pipeline, we made some decisions that may marginally impact fidelity. We believe these greatly improve usability; unmodified OCR remains available in IB-HL for users with different requirements.

Specifically, we remove page numbers and running headers and footers (cf. §4.6, §4.7). These elements are marginalia and have different organizational granularity than the sequence of sentences and paragraphs in the main text. Though marginalia carries useful navigational and bibliographic value, once page boundaries are removed it appears mostly as recurring noise and fragments for the purposes of large-scale computational access. Addressing these forms of paratext (and marginalia more broadly) is left to future work.

Available Compute. The techniques described here are designed to be widely available and reproducible. For both responsible use of resources and reproducibility, we favor frugal computation (Vanderbauwhede 2023), echoing a philosophy of IB-HL. Most of the pipeline runs on CPUs or involves small machine learning (ML) models that run on a single GPU, and we set an approximate target that the whole collection be reproducible within one month on a single NVIDIA DGX Spark.⁴

To run the pipeline, we used the Harvard FASRC Cannon cluster. Knowing we would run this pipeline on a cluster, we designed the pipeline around a distributed computing workflow with systematic error detection and resolution. Ultimately, the *library* powering the pipeline is efficient and modest in its requirements, while the *orchestration* optimizes for large, heterogeneous compute availability.

Throughout, we report the actual compute times required on our cluster or, when relevant, on a single machine. The Harvard Cannon cluster has heterogeneous hardware. During execution, the typical compute node had at least 4 Intel Cascade Lake CPU cores; the typical GPU computation node had at least 4 Intel Sapphire Rapids CPU cores and one NVIDIA A100 80GB GPU. We aim to provide enough information to allow others to assess the cost of reproducing or extending each stage of computation.

⁴This pipeline hits this target with one major exception: computing bits-per-byte for all paragraphs. Though we chose a small 0.6B model for this computation, this step requires months of dedicated GPU time. See §4.11 for details.

4 Processing Pipeline

See Figure 1 for an overview of the processing pipeline. The input is the IB-HL dataset, consisting of 242B o200k_BASE tokens across 983,004 volumes. The pipeline is a sequence of text-to-text transformations. In this section, we describe the aims and methodology of each step; several steps have additional details in Appendix C.

The first step is preprocessing and setup (cf. §4.1). This step also splits the dataset into shards. Most remaining steps share no memory or information across shards and are suited for parallel computation. Each shard goes through Unicode normalization (cf. §4.2), duplicate page removal (cf. §4.3), endmatter separation (cf. §4.4), dehyphenation (cf. §4.5), running header and footer removal (cf. §4.6), page number removal (cf. §4.7), sentence segmentation (cf. §4.8), and subtopic section chunking (cf. §4.9).

By this stage, each volume’s text is organized in subtopic paragraph chunks. We identify (but do not remove) duplicate paragraphs across the whole collection (cf. §4.10) and score every paragraph by its bits-per-byte (BPB) (cf. §4.11). Finally, we detect the language of each paragraph, assemble the metadata, and format the output (cf. §4.12).

4.1 Preprocessing

Later stages in the pipeline apply techniques informed by the output of different base language models. During preprocessing, the pipeline trains the base language models and assembles the data into shards. The pipeline also distills a static multilingual embedding model.

4.1.1 Methodology

Shard Assembly. We group the 983,004 volumes of IB-HL into shards of up to 200 volumes each. Books are grouped into two categories depending on the primary volume language (as determined by IB-HL). One category consists of volumes whose primary language is one of the 138 languages listed in Table 27 in Appendix C.8; we later segment these texts into sentences (cf. §4.8) using the NUPUNKT library (M. J. Bommarito, D. M. Katz, and J. Bommarito 2025) and refer to these as “NUPUNKT-compatible” volumes. The other category consists of all remaining languages. Each shard is assembled from volumes in exactly one category.

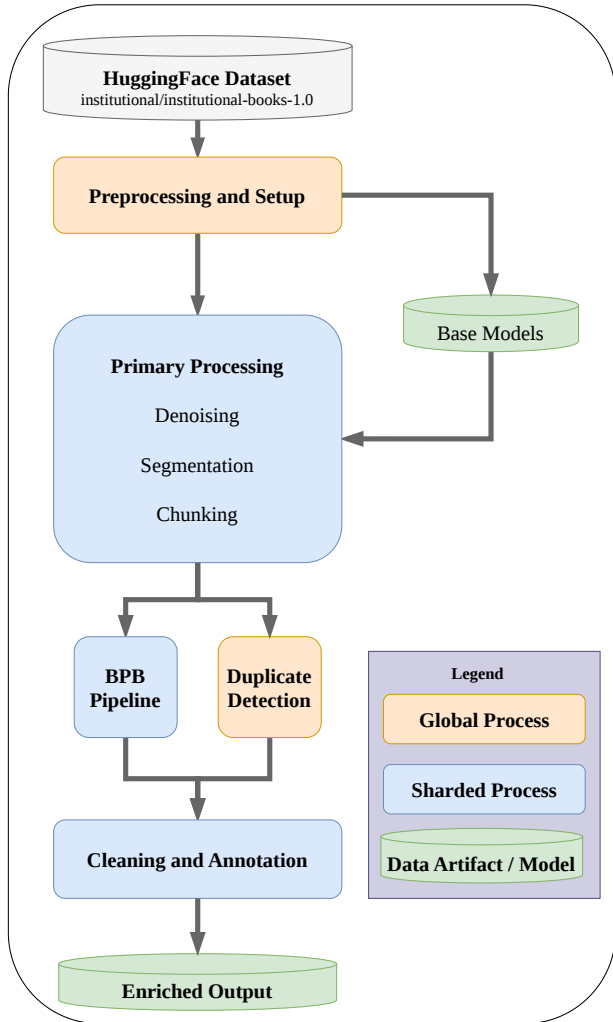


Figure 1: Pipeline Overview

This distinction optimizes for sentence segmentation. NUPUNKT-compatible shards can be processed (up to BPB annotation) without GPUs, whereas the other shards require GPUs for efficient sentence segmentation.

Base Language Models. Base language models are bootstrapped from the corpus itself for improved handling of historical language patterns. We select up to 30 volumes from each language (as identified per volume in IB-HL) when available. Within each language subcollection, we gather n-gram statistics (1-grams to 5-grams) for a small n-gram language model.

N-gram language models are trained on a heavily normalized version of the text: full NFKC Unicode normalization with additional normalizations applied to hyphens, spaces, quotes, and whitespace (cf. Appendix C.2 “hard normalization”). These models are used in dehyphenation (cf. §4.5).

For each of the 138 NUPUNKT-compatible languages, we treat each subcollection as a training corpus for a base NUPUNKT model. Each NUPUNKT corpus undergoes the same Unicode normalization described in §4.2 to match the normalization during inference.

Static Embedding Models. We use MODEL2VEC (Tulkens and van Dongen 2024) to distill BAAI/BGE-M3 (Chen et al. 2024) into a static embedding model. This distilled embedding model is later used in chunking (cf. §4.9). We choose BAAI/BGE-M3 due to its multilingual support. Distilling BAAI/BGE-M3 to a static model using MODEL2VEC allows inference to be performed using only CPUs. Though this lowers precision and accuracy slightly, the gain in computational efficiency enables us to use embeddings throughout the pipeline.

Further, we fine-tune the static embedding model into a static classifier and static subclassifier used for endmatter classification (cf. §4.4). The data used for fine-tuning was synthetically generated and is distributed as GitHub release assets (cf. §4.4.1). We separated the training data into an 80-20 split and oversampled classes during training to balance categories.

4.1.2 Results

Preprocessing lasted 9.5h on 4 cores of a shared Intel Cascade Lake node on a compute cluster. NUPUNKT model training required 3h (averaging ≈ 78.4 s/language). N-gram training required 2.2h (averaging ≈ 31.5 s/language). Distillation required 1h, and training the classifier and subclassifier required 0.5h each. Data transfer occupied the remaining time.

In total, 388 base models and two endmatter classification models were trained during preprocessing:

1. Base n-gram language models (in 250 languages) were trained on subcollections of up to 30 books in each language.
2. Base NUPUNKT segmentation models were trained on subcollections of up to 30 books in each of the 138 NUPUNKT-compatible languages.
3. One endmatter classification model and subclassification model were trained.

Only 82 languages had the full 30 books available in the corpus for base model training. The remaining 168 languages have “base” models built from every book in that language in IB-HL.

The trained endmatter classifier detects endmatter with 0.97 accuracy (F1 0.97). The trained

subclassifier is less accurate, with 0.90 overall accuracy.

4.2 Unicode Normalization

Overcoming the challenges posed by OCR reliability in historical document digitization is an active area of research. Unicode normalization is one basic step towards addressing artifacts and differences in historical typography (Neudecker et al. 2021; Beyene and Dancy 2026).

IB-HL contains volumes spanning centuries of typography. The same underlying text can appear in different byte-level forms. For example, the accented character (é) could be encoded as U+00E9 (é) or U+0065 and U+0301 (e and ´), depending on OCR preference and configuration. OCR treatment is inconsistent across ligatures, diacritics, full-width and superscript text, whitespace, zero-width characters, hyphen and dash code points, curly and straight quotation marks, and scientific or mathematical notation. We follow standard Unicode normalization practices (Whistler 2025; ICU 2020) to facilitate text mining and data preparation.

We choose to prioritize accountability and provenance of the underlying source material: we conservatively normalize text, but do not seek to detect and correct individual OCR mistakes.

4.2.1 Methodology

We have two normalization regimes with different aims: *soft* normalization and *hard* normalization. The exact transformations are in Listing 24 in Appendix C.2.

Soft normalization is conservative. We first apply Unicode NFC composition. The Unicode standard specifies the U+200B zero-width space as a formatting hint for word break and line break opportunities in languages that have no visible word spacing, such as Thai, Myanmar, Khmer, and Japanese (see §23.2 “Layout Controls” in The Unicode Consortium 2022). OCR engines sometimes output zero-width spaces, but these are inconsistent and manifest as invisible noise in textual analysis. We follow the common practice of removing U+200B zero-width spaces. We explicitly preserve *visible* text, including line breaks, curly quotes, hyphens, dashes, ligatures, and accents. Each step resolves encoding-level inconsistencies without altering how the text visibly reads. All output text in IB-HL-ET is soft-normalized.

Hard normalization is lossy and only used internally to determine when two different passages refer to the same reduced text. In addition to the soft normalization’s operations, hard normalization applies NFKC compatibility normalization, maps space-like characters to ASCII space, maps every quote to ASCII quotes, and flattens all tabs and newlines. The output is a single line with ASCII punctuation and NFKC-folded characters. Hard normalization is never stored and never presented to users. It is computed transiently when robustness against typographic variation matters more than fidelity, namely for dehyphenation (cf. §4.5), exact duplicate page removal (cf. §4.3), and duplicate passage identification (cf. §4.10).

4.2.2 Results

A total of 46h (combined wall-time across 1-core jobs on Intel Cascade Lake workstations) was spent on soft-normalizing all the text at the start of the pipeline. Per-volume cost was uniform and inexpensive (≈ 0.17 s/volume).

4.3 Duplicate Page Removal

Within each volume we detect and discard duplicate pages. The vast majority of these examples are repeated scans from the initial scanning process, though some recurring text is also removed. This is distinct from identifying duplicate paragraphs (cf. §4.10).

4.3.1 Methodology

Scanning the same physical page twice usually yields slightly different OCR'd text. Thus we search for *near*-duplicate pages.

Each page is assigned a 128-bit simhash (Charikar 2002) assembled from 9-grams. The simhash is assembled using the MurmurHash3⁵ algorithm (based closely on the original implementation (Appleby 2016)). Pages with fewer than 50 characters are not candidates for duplicate removal to prevent spurious removal of short pages. Initial tests showed that repetitive low-entropy text could dominate the final simhash. For example, pages with the text “.....” have biased simhashes, but are common in tables and lists. To mitigate this problem, we only hash 9-grams with at least 4 distinct characters.

Two simhashes are considered near-duplicate if they differ by at most 6 bits. We compare all qualifying pages' simhashes pairwise within each volume. We group mutually near-duplicate pages with a union-find (as in Chapter 21 of the standard algorithms book by Cormen et al. 2009). Thus if page *A* is a near-duplicate of page *B* and page *B* is a near-duplicate of page *C*, then page *A* is noted as a near-duplicate of page *C* even if their simhashes differ by more than 6 bits. Choosing a 6 bit threshold is conservative and gives a low chance of spurious duplicate removal even after union-find accumulation.

4.3.2 Results

Page-level deduplication required a total of 164h (combined wall-time across 1-core jobs on Intel Cascade Lake workstations), with approximately equal amounts of time spent on (1) transient hard Unicode normalization, (2) simhash computation, and (3) pairwise simhash comparison. We show a few statistics at the top of Table 2.

Approximately 81.6% of books had no duplicate pages. Some books had a substantial portion of duplicate pages. The 10 most extreme are indicated at the bottom of Table 2. These examples have unusually repetitive page scans. The book with barcode 32044081823502 (titled *Memoir of Isaac Richardson*) has 132 pages post-deduplication, while containing 692 page scans (of which only 71 are middlematter).

4.4 Endmatter Separation

A trained page classifier splits each volume into frontmatter, middlematter, and backmatter. We output the separated endmatter largely unprocessed. A trained subclassifier categorizes each endmatter page into one of TOC_INDEX (table of contents or index pages), BIBLIO (bibliography, citations, or lists of references), or OTHERENDMATTER. The remaining pipeline operates primarily on middlematter.

⁵MurmurHash3 is optimized for efficient computation. We capture more of this efficiency by writing a small C++ extension that computes and compares simhashes with MurmurHash3, avoiding the overhead of passing Python objects back and forth. See Appendix C.3.

Books Processed ⁶	983,003
Total duplicate pages removed	1,981,296
Books with at least 1 duplicate page	180,521
Mean pages removed per affected volume	10.98

Barcode	Lang	Pages Removed	Total Pages	% of Book Removed
32044081823502	English	560	692	80.9%
32044097046809	English	338	464	72.8%
32044011481660	English	894	1382	64.7%
32044050769256	French	920	1448	63.5%
32044004373478	English	46	76	60.5%
32044018915694	English	339	566	59.9%
32044011481652	English	721	1238	58.2%
32044088907993	English	720	1256	57.3%
32044085160604	Latin	106	188	56.4%
HL01BW	English	499	886	56.3%

Table 2: Books with High Duplicate Page Rates

4.4.1 Methodology

Reliably detecting endmatter across a diverse collection is known to be challenging. Our approach is modeled on the experimental OCR text post-processing strategy from IB-HL (see §4.9 of Cargnelutti et al. 2025). We use the MODEL2VEC-distilled BAAI/BGE-M3 static embedding model described in §4.1.1 and its static fine-tunes for the classifier and subclassifier.

Synthetic Data. This pipeline uses synthetic data for fine-tuning. On one hand, testing indicates that large-scale annotation of actual endmatter leads to stronger models. On the other hand, testing also indicates that multilingual support is essential for training a multilingual classifier. We deliberately preserve the multilingual nature of the collection, but could not collect high-quality annotations across such a diverse set of languages. Recent research has shown that synthetically generated training data is effective at improving model performance when limited training data is available (Xie et al. 2020; Gunasekar et al. 2023; Allal, Lozhkov, and Bakouch 2024). Further, performance is better for simple, non-subjective tasks after careful curation (Z. Li et al. 2023; Y. Li et al. 2024). Thus using multilingual synthetic data is a pragmatic choice.

To generate synthetic data, we compared the behavior among GPT-OSS-20B (OpenAI 2025), GEMMA3 (Gemma Team et al. 2025), and QWEN3 (Qwen Team 2025). After creating and verifying samples in each of the models, we ultimately choose to focus on GPT-OSS-20B for its performance and permissive licensing. See Listing 26 in the Appendix for the structure of our generation prompts.

We generated 150k training data examples. From these, we selected 44,532 valid training

⁶There is exactly one book in IB-HL that is not in IB-HL-ET. It has the title *Nederlandsche pasicrisie: bevat-tende in alphabetische methode den zakelijken inhoud van alle in Nederland gewezen regterlijke beslissingen en de chronologische lijst dier beslissingen, met opgave van al de verzamelingen van regtspraak waarin zij gevonden worden: alphabetisch gedeelte Chrono. gedeelte: 1887*, barcode HL00NK. This book is omitted because the entire book is one large table with correspondingly poor OCR. The endmatter classifier flagged this book as exceptional as almost the entire book was detected as endmatter.

examples for endmatter-vs-middlematter and 27,192 examples for classifying endmatter. Curation involved several rounds of cleaning and pruning. Generated examples often included markdown; mid-resource languages produced many duplicate examples; and low-resource languages led to vacuous outputs. We stripped markdown, removed duplicates, enforced a minimum length requirement, and checked claimed languages using PYFRANC (pyfranc 2023).

Inference. We treat endmatter separation conservatively and non-destructively. Each book’s detected frontmatter consists of all pages until the first non-endmatter page, and similarly (in reverse) for the backmatter. Pages with only whitespace are not counted as middlematter when determining endmatter boundaries.

The pipeline classifies all pages in each book in batches of up to 1024 pages at a time. This leads to more middlematter classifications than necessary, but most books are classified in a single batch and this leads to little overhead.

Subclassification. We choose to focus on the subtypes TOC_INDEX (Table of Contents), BIBLIO (Bibliography), and OTHERENDMATTER. Both tables of contents and indices tend to have similar grammars and typesetting, making them difficult to distinguish. Bibliographies and lists of references are also easily confused. Other common endmatter pages (dedication pages, publication pages, title pages, copyright pages, bibliographic information, library or collection plates, and so on) vary in form and would require extensive annotation support for accurate prediction.

4.4.2 Results

Endmatter classification lasted 211h (combined wall-time across many 4-core jobs on Intel Cascade Lake workstations). No significant timing differences occurred across languages. This is only computationally viable due to the MODEL2VEC static model architecture.

On average, each book has approximately 14 pages of endmatter. Endmatter pages have additional sources of OCR errors (e.g. hand-written additions, library stamps, book plates), leading to higher variance in classification. The resulting errors more often push endmatter into middlematter than the reverse. For example, one common overly-conservative signal to start endmatter is when a book’s call number is handwritten on some frontmatter page, but OCRred incorrectly.

Classification statistics are in Table 3. Note that the “Total” line does not include blank pages. About 41% of frontmatter and backmatter pages are blank. Backmatter has a higher blank-page rate than frontmatter (47.7% vs. 35.6%), possibly due to fixed folio lengths and trailing sheets.

4.5 Dehyphenation

End-of-line (EOL) hyphens are often introduced to create more consistency in the physical appearance of blocks of text. Removed from their original context, these hyphens are artifacts that can corrupt underlying words (e.g. writing informa- / tion instead of information).

One strategy would be to detect and remove all EOL hyphens using regular expressions. Applying this to the whole dataset, however, would collapse many legitimate words and compound phrases into non-words. We instead resolve end-of-line hyphens using an n-gram language

	Pages		% of all pages
	Frontmatter	7,165,654	1.86%
	Backmatter	6,560,012	1.70%
	All Endmatter	13,725,666	3.55%
	Source Pages	386,256,945	—

	Frontmatter	Backmatter	Total Endmatter	% Non-blank Endmatter Pages
OTHERENDMATTER	3,163,667	1,232,023	4,395,690	54.7%
TOC_INDEX	1,294,656	1,639,797	2,934,453	36.5%
BIBLIO	156,219	556,543	712,762	8.9%
Total	4,614,542	3,428,363	8,042,905	100%
Blank Pages	2,551,112	3,131,649	5,682,761	—

Table 3: Endmatter Classification Statistics

model that ranks possible changes.

4.5.1 Methodology

For every line that ends with a hyphen/dash, we rank three options and choose the most likely:

1. merge the fragments into one (e.g. informa- / tion becomes information),
2. remove the linebreak but keep the hyphen (e.g. mother-in- / law becomes mother-in-law), and
3. replace the linebreak with a space and keep the hyphen (e.g., with interruptions or em-dashes that happen to occur at ends of lines).

Any given word is more likely to appear mid-text than only at line breaks. A probability model trained on the text will capture these likelihoods, and we choose n-gram statistics as our basic model. Ranking is based on character n-grams (using 1-grams to 5-grams). We combine the base n-gram model (trained during preprocessing, cf. §4.1) for the book’s primary language⁷ with a book-specific n-gram model built transiently in memory from the volume’s own text. We only train this book-specific model if the book has end-of-line hyphenation. The base model includes per-language statistical information while the book-specific model includes proper nouns, technical terms, and other book-specific vocabulary.

We estimate the probability of a specific n-gram $w_1^n := w_1 \cdots w_n$ by its add- k smoothed density

$$P(w_1^n) \approx \frac{(\#w_1^n) + k}{(\#\text{n-grams seen}) + k \cdot (\text{vocab size})}$$

with $k = 0.001$. This approach is modeled after standard n-gram language models (see for instance Chapter 3 of Jurafsky and Martin 2026), but with some simplifying assumptions for speed. The add- k smoothing is a crude form of Laplace smoothing and avoids exact 0. To determine the relative ranking among the three possibilities, we approximate the joint probabilities of each option. We estimate the joint probabilities of each possibility using a 10-character window on each side of the hyphen.

⁷As detected by IB-HL. If the language is “unknown”, we default to English, the most common language in IB-HL.

When an n-gram w_1^n has never been seen, we approximate the probability using the so-called *Stupid Backoff* (Brants et al. 2007) algorithm with backoff probability of 0.4 times the $(n - 1)$ -gram formed when omitting the first character,

$$P(\text{unseen } w_1^n) \approx 0.4 \cdot P(w_2^n).$$

The resulting language model is not a true probability distribution, but it adequately ranks the three possibilities and is faster than more sophisticated models.

Overall, the pipeline computes these two sets of probabilities using both the book-specific model and the base language models, computes associated perplexities, and weighs both equally for the final determination. (See Appendix C.5 for discussion of multiple other technical details and alternatives considered. See Appendix D for an example where dehyphenation performs poorly on technical text).

4.5.2 Results

		High Dehyphenation Languages	
		Language	Avg Removed/Page
Total Hyphens Removed	2,322,246,287	Latin	11.3
Estimated EOL Hyphen Removal Rate	$\approx 89\%$	Russian	11.1
Estimated EOL Hyphen Keep Rate	$\approx 7\%$	Bulgarian	11.1
Estimated EOL Hyphen+Space Rate	$\approx 4\%$	Polish	10.3
		Hungarian	9.3

Table 4: Hyphenation Statistics

A total of 640h (combined wall-time across many 4-core jobs on Intel Cascade Lake workstations) was spent on dehyphenation. Run statistics are in Table 4. Typically (89% of the time) the dehyphenation strategy removed the hyphen and newline. This supports the widespread practice of using regular expressions to detect and remove all end-of-line hyphens, but also suggests that this practice is not always appropriate.

Cyrillic, Latin, and European languages have by far the most inserted end-of-line hyphens. In this collection, Latin has the highest per-page rate, consistent with dense scholarly printing of classical text. For English books, the average number of hyphens merged per page was 4.9, substantially smaller than Latin or the Slavic languages. As expected, many languages have almost no end-of-line hyphens. Chinese, Japanese, and Arabic all average below 0.1 removed hyphens per page (and those with hyphens are often multilingual).

4.6 Running Header and Footer Removal

Printed books often show a running header and/or footer containing the title, author, or chapter name. OCR tends to place these among the first or last lines on a page. Removing page boundaries causes these to become inline noise, disrupting paragraphs that span multiple pages. We identify recurring short lines near the top or bottom of nearby pages and remove them.⁸

⁸This removal strategy is not perfect. See Appendix D for a concrete example where this strategy removes too much text. Improved marginalia handling is left to future work.

4.6.1 Methodology

We identify running headers and footers by detecting locally concentrated clusters. We allow *near*-duplicates instead of *exact* duplicates as page numbers are often appended to running headers or footers by OCR; but we set the threshold conservatively to minimize excess removal. The top or bottom 5 lines of each page form the initial candidates for removal. We choose the top and bottom 5 lines because OCR engines are not always consistent with placement of marginalia and edge-matter; further, some books have multiline headers and footers.

For each of the top and bottom 5 lines of each page, we strip whitespace and discard candidates shorter than 6 characters. Lines this short are not likely to be a running header/footer and are too short to match reliably. Each remaining candidate is represented as a MinHash (Broder 2000) signature made from 128 permutations over character 5-grams. Each signature is indexed in a MinHash locality-sensitive hashing (LSH) index⁹ with Jaccard threshold 0.85 (see for example Chapter 3 of Leskovec, Rajaraman, and Ullman 2020; we use the implementation in the DATASKETCH library by Zhu et al. 2024). To be marked as a duplicate, a line must have at least 3 near-duplicates within a 5-page span.¹⁰ These marked duplicates (in the top or bottom 5 lines, near-exact threshold, with at least 3 duplicates in a cluster) are deleted from the middlematter.

4.6.2 Results

Running header and footer removal required 927h (combined wall-time across many 4-core jobs on Intel Cascade Lake workstations) at an average of 3.4s/volume. Removal statistics are given in Table 5. Random sampling shows that languages with vertical reading orientation

Total Header and Footer Lines Removed			272,583,638		
Volumes with ≥ 1 Removal			760,238 (77.3%)		
Average Removed/Volume			277		

High Removal Languages			Low Removal Languages		
Language	Avg/Book	Avg/Page	Language	Avg/Book	Avg/Page
Scots	233	0.93	Mandarin Chinese	7.9	0.01
English	347	0.91	Slovak	42.4	0.12
French	289	0.66	Ukrainian	38.8	0.15
Greek	272	0.65	Arabic	86.1	0.26
Latin	221	0.59	Gaelic	48.3	0.27

Table 5: Header and Footer Removal Statistics

have less accurate header/footer detection. This appears to be primarily due to placement within the OCR text: while we observed that headers and footers usually appear at the top or bottom of OCRed pages, language-specific failure modes of OCR engines may lead to different, hard to anticipate placements.

⁹The headers and footers are kept in separate LSH indices so that headers only match with other headers and footers only match with other footers.

¹⁰Headers and footers often differ between odd and even pages. To lower random matching, we require not one repetition, but at least two. This explains the requirement for ≥ 3 instances within a 5-page span.

4.7 Page Number Removal

As with running headers and footers, page numbers appear in margins and result in artifacts at almost every page break. OCR engines typically place page numbers near the top or bottom of each page. We look for short, numeric lines at the top and bottom of pages and remove them.

4.7.1 Methodology

We detect a line as a page number if

1. it is in the top or bottom 5 lines of OCR output,
2. it has at most 8 characters,
3. at least one character is *numeric*, and
4. at most one non-whitespace character is *non-numeric*.

We consider a character *numeric* if, after NFKC Unicode normalization, the character is in the numeric Unicode category (specifically if `unicodedata.category(ch).startswith('N')` is true in Python).

In addition, there is one later round of *stray number removal* that occurs after separating the text into sentences. Any “sentence” that consists of a single short number (with at most one non-numeric character after removing white-space) is removed.

4.7.2 Results

Page number removal required a total of 2.5h (combined wall-time across many 4-core jobs on Intel Cascade Lake workstations) at an average of less than 0.01 s/volume. This is expected: page number detection is a short sequence of string inspections with no additional models or hashing. Statistics for the run are given in Table 6. We observe that the median removal

Total Page Numbers Removed	433,722,448
Volumes with ≥ 1 Removal	980,441 (99.7%)
Average Removed/Volume	441
Average Removed/Page	1.123
Median Removed/Page	1.064

Table 6: Page Number Removal Statistics

is close to one number per page, agreeing with expectation. A common source of additional removals comes from footnote and caption markers, which are often placed on their own lines by OCR engines.

The average is larger than the median. This reflects a long tail: 13k books had ≥ 3 page numbers removed per page. The highest ratio is 8.7 removed “page numbers” per page, held by the book with barcode HN3J4P, titled *The American ready reckoner: designed to insure correctness as well as despatch in business*. This book has a multiplication table on almost every page. See Figure 7 for an example portion. Currently available OCR text in IB-HL sometimes places every cell of a table on its own line. Inspection shows that all top 10 books are similarly full of tables.

	19	19 $\frac{1}{4}$	19 $\frac{1}{2}$	19 $\frac{3}{4}$	19 $\frac{1}{2}$	19 $\frac{3}{4}$
1	,19	,19 $\frac{1}{4}$	,19 $\frac{1}{2}$	,19 $\frac{3}{4}$	,19 $\frac{1}{2}$	,19 $\frac{3}{4}$
2	,38	,38 $\frac{1}{2}$	,38 $\frac{1}{2}$	,39	,39 $\frac{1}{2}$	,39 $\frac{1}{2}$
3	,57	,57 $\frac{3}{4}$	,58	,58 $\frac{1}{2}$	,59	,59 $\frac{1}{4}$
4	,76	,77	,77 $\frac{1}{2}$	,78	,78 $\frac{3}{4}$	,79
5	,95	,96 $\frac{1}{4}$	,96 $\frac{3}{4}$	,97 $\frac{1}{2}$	,98 $\frac{1}{4}$	,98 $\frac{3}{4}$
6	1,14	1,15 $\frac{1}{2}$	1,16	1,17	1,18	1,18 $\frac{1}{2}$
7	1,33	1,34 $\frac{1}{4}$	1,35 $\frac{1}{2}$	1,36 $\frac{1}{2}$	1,37 $\frac{3}{4}$	1,38 $\frac{1}{4}$
8	1,52	1,54	1,54 $\frac{3}{4}$	1,56	1,57 $\frac{1}{2}$	1,58
9	1,71	1,73 $\frac{1}{4}$	1,74	1,75 $\frac{1}{2}$	1,77	1,77 $\frac{3}{4}$
10	1,90	1,92 $\frac{1}{2}$	1,93 $\frac{1}{2}$	1,95	1,96 $\frac{3}{4}$	1,97 $\frac{1}{2}$
11	2,09	2,11 $\frac{3}{4}$	2,12 $\frac{3}{4}$	2,14 $\frac{1}{2}$	2,16 $\frac{1}{2}$	2,17 $\frac{1}{4}$
12	2,28	2,31	2,32	2,34	2,36	2,37
13	2,47	2,50 $\frac{1}{4}$	2,51 $\frac{1}{2}$	2,53 $\frac{1}{2}$	2,55 $\frac{3}{4}$	2,56 $\frac{3}{4}$
14	2,66	2,69 $\frac{1}{2}$	2,70 $\frac{3}{4}$	2,73	2,75 $\frac{1}{2}$	2,76 $\frac{1}{2}$
15	2,85	2,88 $\frac{3}{4}$	2,90	2,92 $\frac{1}{2}$	2,95	2,96 $\frac{1}{4}$
16	3,04	3,08	3,09 $\frac{1}{4}$	3,12	3,14 $\frac{3}{4}$	3,16
17	3,23	3,27 $\frac{1}{4}$	3,28 $\frac{3}{4}$	3,31 $\frac{1}{2}$	3,34 $\frac{1}{2}$	3,35 $\frac{3}{4}$
18	3,42	3,46 $\frac{1}{2}$	3,48	3,51	3,54	3,55 $\frac{1}{2}$
19	3,61	3,65 $\frac{3}{4}$	3,67 $\frac{1}{2}$	3,70 $\frac{1}{2}$	3,73 $\frac{3}{4}$	3,75 $\frac{1}{4}$

Figure 7: Part of a page from HN3J4P, *The American ready reckoner*, showing how to multiply 19, 19 $\frac{1}{4}$, and so on by numbers from 1 to 19. Every table entry receives its own line in the OCRd output, so almost every checked line resembles a page number.

4.8 Sentence Segmentation

We now reorganize the middlematter into sentences. The corpus spans roughly 250 languages with different punctuation conventions and scripts, so we use multiple sentence segmentation strategies.

For languages with English-like punctuation, we use NUPUNKT (M. J. Bommarito, D. M. Katz, and J. Bommarito 2025), a modern implementation of the Punkt algorithm (Kiss and Strunk 2006). NUPUNKT is an unsupervised algorithm that attempts to recognize sentence-ending punctuation and common ways for that same punctuation to occur without ending a sentence (such as in abbreviations like Dr., U.S., etc.).

For other languages, we instead use WTPSPLIT and SAT (Segment-any-Text, by Frohmann et al. 2024), a multilingual sentence segmentation system. Specifically, we use the SAT-3L-SM¹¹ three-layer general sentence segmentation model for its balance of speed and performance.¹²

Determining when to use NUPUNKT and when to use SAT is not straightforward and is described in more detail in Appendix C.8. The split is heavily skewed: 97.9% of books go through the NUPUNKT engine and the remaining 2.1% go through the SAT engine.

¹¹<https://huggingface.co/segment-any-text/sat-3l-sm>

¹²As recommended by SAT, see <https://github.com/segment-any-text/wtpsplit#available-models>.

4.8.1 Methodology

For the 138 NUPUNKT-compatible languages listed in Table 27 in the Appendix, the sentence segmentation process starts with a base corpus generated during preprocessing (cf. §4.1) and adapts it for each book. In Punkt-based algorithms, adaptation involves combining abbreviation lists and updating punctuation neighborhood counts.

Combined base model training for NUPUNKT required 3 hours (on 4 cores of a shared Intel Cascade Lake cluster node). It is possible to apply the base language NUPUNKT models directly, without per-book adaptation, but we found that performance improved with per-book adaptation. This was especially true on books with many acronyms, abbreviations, and other technical non-sentence-terminating punctuation. This comes with significant cost: per-book adaptation requires approximately 16× more compute time than using only the base model to segment the book.¹³ (Concretely: segmenting a book using a loaded model requires on average 0.6s on a single core of an Intel Cascade Lake workstation; adapting a single book requires just over 10s on average).

Punkt-based algorithms do not apply to every language, especially when punctuation differs strongly from English. Determining when NUPUNKT applies is complicated by the continued incorporation of Western punctuation symbols into writing systems (see e.g. J. K. Lee 2014; Twine 1984). Similar texts on similar subjects in the same language can have different punctuation standards. For the 112 languages that we do not consider NUPUNKT-compatible, we instead use the three-layer general segmentation SAT neural network-based sentence segmenter. The SAT model is the only step in the general pipeline that requires a GPU. On an NVIDIA A100 GPU, segmenting a typical 400 page book takes 1.18s on average.

Thus SAT requires twice as much time as applying the base NUPUNKT model, but is *faster* than applying per-book NUPUNKT adaptation. On the other hand, SAT requires a GPU.

4.8.2 Results

Sentence segmentation required a total of 2855h (combined wall-clock time on our distributed cluster; heterogeneous nodes). See Table 8 for timing by type. NUPUNKT adaptation and inference were performed on single-core Intel Cascade Lake workstation nodes. SAT inference was performed on heterogeneous nodes containing at least 4 Intel Sapphire Rapids CPU cores and 1 NVIDIA A100 GPU.

Engine	Volumes	Total Time	Avg s/vol
NUPUNKT Per-Book Adaptation	962,373	2,691h	10.06s
NUPUNKT Inference	962,373	157h	0.59s
SAT (GPU)	20,629	6.6h	1.18s

Table 8: NUPUNKT and SAT Timing Information.

Output statistics are given in Table 9.¹⁴ NUPUNKT sentences and SAT sentences have approximately the same number of characters, but SAT sentences tend to have far more words. Whether this reflects language conventions or segmenter differences is unclear.

¹³ For this reason, the pipeline library allows simple configuration to enable or disable per-book NUPUNKT adap-

Engine	Sentences	% all sentences	Avg chars/sentence	Avg words/sentence
NUPUNKT	7,010,365,214	99.00%	113.2	23.3
SAT	70,838,120	1.00%	119.5	36.7

Table 9: Sentence Segmentation Statistics

4.9 Topical Chunking

The next step of the pipeline is to organize the sequence of sentences into “subtopic paragraphs” and “subtopic sections”. Our motivation and terminology stem from Hearst 1997, which characterizes a text structure as a sequence of subtopics within broader main-topic discussions. We use “subtopic paragraph” and “subtopic section” to mean contiguous stretches of text that are *about* something. A single volume may contain many subtopic sections, and a single subtopic section may contain many subtopic paragraphs.

This differs from the actual presentation within each volume. Organizing sentences into hierarchical semantic sections is not how authors typically organize their writing. To paraphrase Hearst, many volumes consist of long sequences of paragraphs without any structural demarcation. Organizational conventions change over time and vary across languages. We expect a more consistent semantic structure to be valuable for downstream tasks. Where sentences are too small and complete volumes are too big, subtopic paragraphs and sections are self-contained and topically coherent. Subtopic chunks are natural for context-aware search, semantic search and retrieval, long-context focused LLM training, duplicate detection, and other information retrieval tasks.

Our approach to topical chunking is a modern instantiation of Hearst’s TextTiling algorithm. But where TextTiling detects topical shifts from a variety of lexical patterns, we search for dips in cosine similarity between static embeddings of sentences.

4.9.1 Methodology

We use the MODEL2VEC static model distilled from BAAI/BGE-M3 (cf. §4.1) to generate static embeddings of sentences. The rest of the approach is strongly informed by TextTiling (Hearst 1997). Concretely:

1. Embed each sentence using a static model distilled from BAAI/BGE-M3 (in batches of 256 sentences).
2. Score each sentence-gap by semantic cohesion. Take the mean embedding of the previous 5 sentences and the mean embedding of the next 5 sentences, and compute their cosine similarity. High scores suggest topical continuity. Low scores signal topical shift.
3. Collect the sequence of gap scores into a similarity curve. We apply a single-pass moving average (width 3) to reduce high-frequency oscillation from single-sentence noise.
4. Compute the TextTiling valley-depth at each gap. At each sentence-gap, we find the previous and subsequent peaks (where sentences are most topically cohesive) and sum the differences,

$$\text{valley depth} = (\text{left_peak} - \text{gap_value}) + (\text{right_peak} - \text{gap_value}).$$

tation.

¹⁴The number of words per sentence was computed for this table using the POLYGLOT library.

- Deep, two-sided valleys correspond to strong topic boundaries. Shallow valleys do not.
5. Greedily select sentence boundaries. A sentence-gap is a boundary candidate if its depth exceeds a per-book threshold ($\text{mean}(\text{valley_depth}) - 0.5\text{std}(\text{valley_depth})$). Candidates are taken greedily from deepest to shallowest, subject to the requirement that no segment be shorter than 3 sentences.¹⁵
 6. Use the accepted gaps as subtopic-paragraph boundaries.

Our methodology deviates from Hearst in two places: we use static sentence embeddings instead of term-frequency overlap and lexical statistics since sentence embeddings better capture semantic content; and we operate on whole sentences instead of fixed-length pseudosentences since sentence boundaries are already provided by the segmentation step in the pipeline.

Section-level chunking reuses *exactly* this procedure, except using the identified subtopic paragraphs in place of sentences. Iteration would yield larger tiers in the semantic hierarchy.

4.9.2 Results

A total of 1,568h was required to perform subtopic chunking (combined wall-time across many 4-core jobs on Intel Cascade Lake workstations) with an average of 5.7 s/volume. The median time per book was 1.5 s/volume and the overall time was heavily right-skewed by long books with many sentences. For a volume with T tokens, N sentences, and B candidate paragraph breaks, our implementation runs in time $O(T + N \log N + B^2)$; the terms come from embedding cost, sorting sentence gaps, and greedily building the boundary list, respectively. In practice, $B \approx N/5$, hence long books with many sentences will take a disproportionate amount of time. Core statistics are in Table 10. The average number of sentences per paragraph is consistent

Total Sentences	7,081,203,334
Detected Subtopic Paragraphs	1,394,049,896
Detected Subtopic Sections	297,350,019
Avg Paragraphs per Section	4.688
Avg Sentences per Paragraph	5.080

Table 10: Topical Chunking Statistics

across languages: the smallest is ≈ 4.8 in Japanese, while the largest is 5.1 in Latin. No significant difference in numbers of sentences exists between volumes segmented using NUPUNKT and volumes segmented using SAT.

The distribution of the lengths of paragraphs is shown in Figure 11. The plot shows a roughly log-normal, unimodal distribution. The 3-sentence minimum manifests as the relatively sharp lower bound. There are 950 paragraphs with over 100k characters, mostly from poor OCR on tables.

4.10 Duplicate Identification

Chunking the text into semantic subtopic paragraphs makes it easier to identify duplicate text. We identify near-duplicate subtopic paragraphs across the whole dataset. In general, they are

¹⁵In the greedy selection algorithm, the last chunk in the book is not constrained to have at least 3 sentences. The final chunk in each book may be smaller.

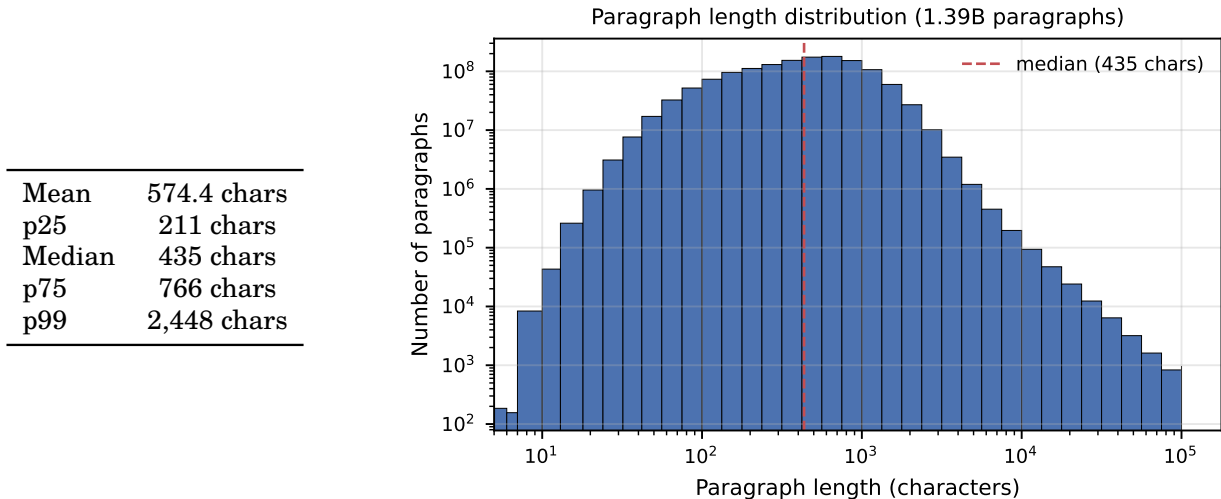


Figure 11: Subtopic Paragraph Length Distribution

not deleted¹⁶ and are instead annotated and left in place.

Our treatment contrasts with common practice. Large-scale dataset preparation pipelines for language model training routinely perform exact and fuzzy deduplication to remove repeated documents (Brown et al. 2020; Gao et al. 2020). Some recent work suggests more rather than less aggressive deduplication (K. Lee et al. 2022), and that even semantic near-duplicates can be removed with little impact on downstream performance (Abbas et al. 2023).

Though removing duplicate paragraphs might make sense for some tasks, it does not work for all problems. Instead, we mark paragraphs with duplicates so that downstream users can determine whether to use or discard them as appropriate.

4.10.1 Methodology

Our approach to duplicate identification follows other large-scale deduplication efforts and reuses several methodological components from page deduplication (cf. §4.3). We follow a typical simhash and LSH strategy over entropy-filtered 9-grams hashed with MurmurHash3 and consider two simhashes duplicates if they differ in at most 5 bits.

We compute 128-bit simhashes (Charikar 2002) over 9-grams from hard-normalized (cf. §4.2), lower-cased text. Simhashes are only computed over 9-grams that contain at least 4 distinct characters to avoid noise from strings of repeated text. Each 9-gram is hashed with MurmurHash3 and accumulated into a standard simhash.¹⁷

Thus each of the ≈ 1.4 B paragraphs is reduced to a 128-bit signature. Two simhashes that differ by at most 5 bits are considered duplicates. This is conservative but allows small typographic deviations or OCR artifacts to not disrupt duplicate identification. The naive strategy of pairwise comparison (as executed with page deduplication) is no longer viable. In-

¹⁶One exception comes from library boilerplate. Two clusters consisting of library fine information are removed, as described in the Results section §4.10.2.

¹⁷We again use a small C++ extension for computing simhashes. See Appendix C.3.

stead, to identify these duplicate simhashes, each 128-bit signature is split into 6 bands (with (22, 21, 21, 21, 21, 22) bits, respectively). If two signatures differ by at most 5 bits, then they must exactly agree on at least one band. Thus we seek signatures that exactly match at least one band.

We simultaneously determine all matches in a band by sorting a document with lines of the form (band_value) | doc_id. Each contiguous range of identical band values is one bucket of candidates to check for duplicates. We create 6 documents (one for each band) and sort each individually. The natural unique identifier for each document is its barcode.paragraph_index. As many barcodes are 14-character strings, if we used this identifier then the band-file to sort would have size

$$\approx 1.4 \times 10^9 \times (3 \text{ bytes} + 18 \text{ bytes}) \approx 30\text{GB}.$$

This would be larger than necessary and negatively affect sort time. Instead, we construct a new document id for efficient (and predictable) byte packing.

After creating simhashes for all paragraphs, we read these simhashes in a fixed order and assemble

1. A global binary simhash array hashes.bin, the global concatenation of all paragraph simhashes. (1.4B paragraphs, each with a 16-byte simhash, yielding a file of size $\approx 23\text{GB}$)
2. An auxiliary array book_ids, a list of volume barcodes in simhash-reading-order. (1M strings, tens of MB in size)
3. An auxiliary array book_offsets, one uint64 per volume giving the index in hashes.bin at which that volume's paragraphs begin. (1M uint64 integers, 8MB in size).

These are constructed in order, so book_offsets strictly increases and allows one to detect both book starts and book ends; and the n th entry in book_offsets corresponds to the n th barcode in book_ids. With these arrays, the global document index of a paragraph is *its index in the global simhash array*. Binary search on book_offsets allows recovery of the barcode and paragraph index. We add a byte-packing assumption that the document index can be specified by one uint32 for minor convenience (capping tractable corpus size to $2^{32} \approx 4.29\text{B}$ paragraphs).

Thus each paragraph is uniquely specified by a 4-byte doc_id. We set up band sorting by packing each (band value, doc_id) into a single uint64 key

$$\text{key} = (\text{band_value} \ll 32) \mid \text{doc_id}.$$

The band-files for sorting are thus each about 11GB in size and consist of 64-bit integers for sorting. These can be sorted in memory and a single linear scan recovers all candidate buckets.

To process the buckets in parallel, we memory-map the global simhash array hashes.bin. A pool of workers computes exact pairwise Hamming distances using two xors, two popcnts, and an addition to combine. Positively identified results are added to a global union-find data structure, which ultimately yields the clusters.

After clusters are identified, a unique cluster_id is assigned (of the form barcode:par_idx, chosen as the alphabetically first barcode of a volume in the cluster). During annotation, every paragraph in a cluster is annotated with one of two tags: the paragraph that was chosen as the cluster_id receives the annotation

```
<p data-clusterid="BARCODE:PAR_IDX" data-representative>
```

...

and non-representative paragraphs are wrapped in an aside tag

```
<aside data-cluster="BARCODE:PAR_IDX">
<p> ... </p>
</aside>
```

We describe this more when discussing annotation in §4.12.

4.10.2 Results

Duplicate paragraph identification required 129h (127h of combined wall-clock time across distributed nodes with 4 Intel Cascade Lake cores to compute simhashes, and 1.5h on a 16-core MacBook Pro M4 Max for simhash comparison). The statistics in Table 12 show that 1,932,557 same-book duplicate clusters were detected, *fewer* than the number of removed duplicate pages (1,981,296 as discussed in §4.3.2; removing duplicate pages thus significantly reduced unnecessary duplicate identification). Though some volumes repeat paragraphs internally, most detected duplicates span multiple volumes.

Identified Duplicate Counts		Cluster Size	# Clusters
Total Subtopic Paragraphs	1,394,049,896	2	40,123,977
Duplicate Representatives	51,233,629	3	7,077,321
Duplicate Non-Representatives	72,856,480	4	2,119,430
Average Cluster Size	2.42 paragraphs	5–10	1,725,371
Single-book Clusters	1,932,557 (3.8%)	11–100	186,069
Multi-book Clusters	49,301,072 (96.2%)	101–1000	1,376
Books With Identified Duplicates	557,961 (56.8%)	1000+	85

Table 12: Duplicate Identification Statistics

Language	# Duplicate Paragraphs	% of All Paragraphs in Language
English	57,815,405	8.09%
German	5,976,526	2.41%
French	5,804,669	3.07%
Latin	957,619	2.26%
Italian	640,421	1.38%
Spanish	475,111	1.80%
Dutch	219,803	1.32%
Russian	153,856	0.61%
Hungarian	47,632	0.75%
Hebrew	39,949	1.04%

Table 13: Language Distribution of Detected Duplicate Paragraphs

Table 13 shows English as a pronounced outlier in duplication rate, at more than 2.5 times the rate of the next language. This likely reflects the size of the English subcollection: with 487k English volumes, there is a bigger pool of potential duplicate books (including reprints, reorganized collections, etc.).

The average cluster consists of ≈ 2.42 duplicates, but the distribution is long-tailed. We found

85 clusters containing over 1000 different duplicates. Inspection shows that these exceptionally large clusters tend to be either missed boilerplate or poorly-OCR'd tabular filler. The two largest were variations of the same library fine description:

```
REP 32044004341889.672 - 21,056 Copies
REP 32044004442943.3353 - 19,072 Copies
-----
This book should be returned to the Library on or before the
last date stamped below. A fine of five cents a day is incurred
by retaining it beyond the specified time. Please return promptly.
```

As these were missed boilerplate (and were always the exact last paragraph in the book, from the back cover), we removed these paragraphs from the dataset.

The next several largest clusters are all tabular noise:

```
REP 32044000056234:311 | 16,954 | Do. Do. Do. Do. Do. Do. Do. Do. Do.
REP 32044001172907:42  | 16,804 | .....
REP 32044004599593:258 | 14,942 | Dollars. Dollars. Dollars. Dollars.
REP 32044010095289:65  | 13,360 | idem. idem. idem. idem. idem. idem. idem.
REP 32044002052694:1091 | 12,708 | do. do. do. do.
REP 32044004778643:383 | 9,033 | Value. Quantity. Value.
REP 32044004365193:2145 | 7,896 | Do. Do. Do.
REP 32044004554093:4597 | 7,262 | ..do. .do. .do. ..do.
```

The Do. Do. repetitions are shorthand for “ditto”, the same as idem. idem. The repeated Dollars. and Value. Quantity. lines are orphaned lines from tables.

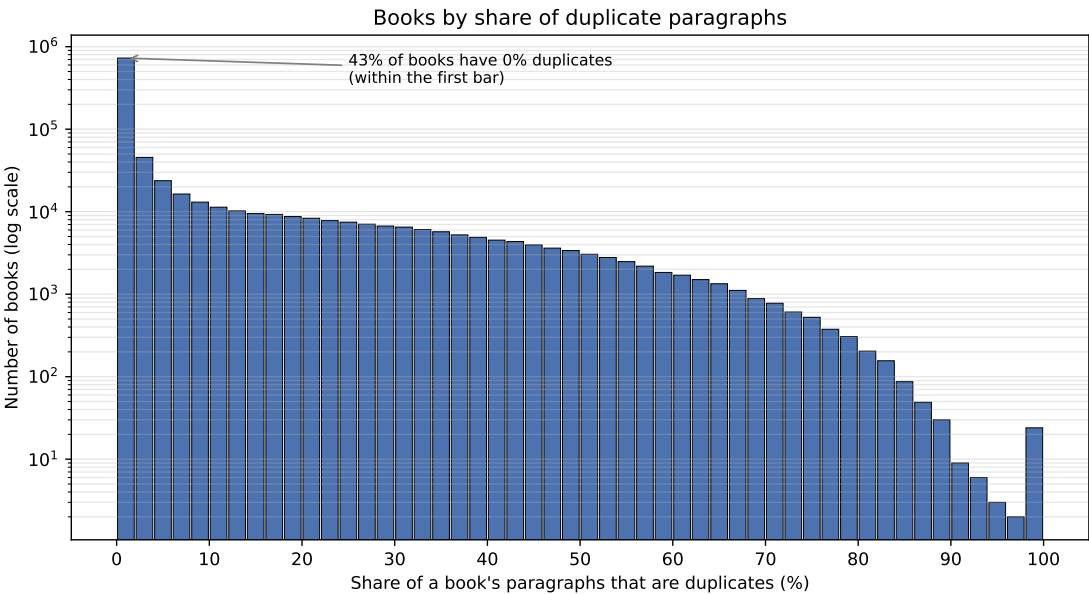


Figure 14: Books by % Duplicate Subtopic Paragraphs

Figure 14 shows the number of books binned by the fraction of identified duplicate subtopic paragraphs. About 43% of books have no duplicate paragraphs and 80% are under 5%. The figure shows a smooth S-curve of decay, except for 24 books with 98–100% duplication. Inspection shows that these 24 books are reprintings, edition changes, or duplicate books in

the collection. For example, the three volumes with barcodes HWJX6S, HWJX7H, HWJX63 all contain a play *The Pigeon* by John Galsworthy, which is included in the 3-play anthology 32044086883501.

4.11 Bits-Per-Byte

OCR quality over a heterogeneous, multi-century, multilingual collection produces text of varying data fidelity. Earlier stages of the pipeline have shown that OCR quality is generally high. But we have also observed that tables and marginalia are often poorly OCRed and yield noisy paragraphs in this collection.

We seek a computable, language-agnostic number per paragraph that:

1. ranks paragraphs by how predictable, fluent, and well-formed their text is,
2. can be summarized per book as a quality statistic, and
3. facilitates downstream filters that keep typical prose and omit unusual extremes.

One approach is to use language model *cross-entropy*. It is common to normalize by the number of tokens and report the exponentiated value as *perplexity*, which has a long history as a corpus-filtering signal (e.g. Wenzek et al. 2020, Gao et al. 2020, and Marion et al. 2023). But the numeric scale of perplexity depends both on the underlying model tokenizer and the language of the text; tokenizers can produce different numbers of tokens for different scripts as a result of different priorities and availability of training data. We instead normalize by bytes: the Bits-Per-Byte (BPB) measures how many bits the model needs to encode each byte of text, which is closer to tokenizer-independent, language-independent data.

Computing BPB is more expensive than the rest of the pipeline and requires dedicated GPU resources. We think of computing BPB as an optional phase performed after the primary processing pipeline. Our implementation allows simple configuration to skip BPB evaluation. But as per-paragraph BPB enables users to filter or seek out-of-distribution text without needing to perform these computations themselves, we believe this is a valuable contribution.

4.11.1 Methodology

For each subtopic paragraph, we compute its BPB with a causal language model and accumulate the next-token log-likelihood it assigns, then divide by byte count:

$$\text{BPB} = \frac{1}{\ln 2} \cdot \frac{-\sum_t \ln p_\theta(x_t | x_{<t})}{(\text{number of UTF-8 bytes})}.$$

We use QWEN/QWEN3-0.6B-BASE (Qwen Team 2025) as the reference model as it has strong multilingual support. We use a relatively small 0.6B parameter model so that it is computationally possible to compute BPB for all 1.4B paragraphs in the collection.¹⁸ The numerator is exactly the model’s total negative log-likelihood of the paragraph. Lower BPB means the paragraph is more predictable; higher BPB means it is more surprising.

Prior to computing BPB, we sort all paragraphs in each book by length and create batches of 16 paragraphs that all have approximately the same length. As shown in Figure 11, the distri-

¹⁸In Appendix C.11 we show that QWEN/QWEN3-0.6B-BASE is an effective proxy for approximating perplexity and BPB for larger QWEN models.

bution of paragraph lengths is roughly log-normal with a long tail; thus presorting prevents large amounts of unnecessary padding in the computation.

In our implementation, excessively short and excessively long paragraphs are each assigned a sentinel value of -1 rather than a score. Paragraphs with ≤ 4 characters are too short for meaningful contextual prediction. And paragraphs longer than 14,134 tokens also receive the -1 sentinel value. This is due to our implementation encountering a 32-bit index limit in the PyTorch CUDA kernel. The number of logits produced from a paragraph is `num_tokens x vocab_size` and must have an `int32` address. The `vocab_size` in our model is 151,936, leaving $(2^{31} - 1)/151936 \approx 14134$ as the max size. In addition, if the BPB computation leads to a GPU Out-Of-Memory error, the paragraph is assigned a -1 sentinel value. This latter case occurred on 187k paragraphs during the initial pipeline run; these were resolved by recomputation and the released dataset contains no such cases.

We compute some summary statistics about the BPB distribution across subtopic paragraphs for each book: the min, max, mean, median, and the 10th, 30th, 70th, and 90th percentiles. Each of these statistics ignores paragraphs with the sentinel value -1 . Further, in the final dataset we do not include BPB annotations with the sentinel value.

We compute these statistics at the book level because absolute BPB values depend strongly on genre and technical material. BPB should be interpreted as a measure of predictability and not of value. Legitimate-but-unusual content, including mathematical notation or low-resource languages underrepresented in the language model’s training, will be “surprising” to the model and have a high BPB value. By providing per-book statistical distributions, we encourage relative filtering instead of establishing corpus-wide cutoffs.

4.11.2 Results

Computing BPB for all 1.4B paragraphs in the collection required a total of ≈ 2000 GPU-hours (≈ 83 GPU days, combined wall time over hundreds of GPU nodes on a distributed, heterogeneous cluster; each node had one NVIDIA A100 GPU and at least 8 Intel Sapphire Rapids CPU cores) with an average of ≈ 7 s/volume or $\approx 700k$ paragraphs per GPU-hour.

Subcollection	# Subtopic Paragraphs	Mean	p25	Median	p75
All [†]	1,393,997,373	1.666	1.179	1.510	2.033
NUPUNKT	1,379,804,105	1.661	1.178	1.506	2.026
SAT	14,193,268	2.179	1.532	2.076	2.824

[†]The 1,394,049,896 subtopic paragraphs in Table 10 are reduced to 1,394,009,768 in the released dataset after removing 40,128 paragraphs in the two deleted library-fine boilerplate clusters (§4.10.2). Of these, 1,393,997,373 receive a BPB score and 12,395 keep the -1 sentinel. This leads to the small differences in totals in these tables.

Table 15: Core BPB Statistics

See Table 15 for statistics on the distribution. NUPUNKT-compatible languages have smaller BPB on average. One possible explanation for this difference is that NUPUNKT-compatible languages (such as English) form the bulk of the pre-training data for the QWEN3 language model. A second possibility is that currently available OCR is more accurate on NUPUNKT-compatible languages than on languages with different scripts and punctuation.

The single paragraph with the smallest (valid) BPB is from a book of patent records and is

Alphabetical list of patentees—Continued.

Name, residence, and invention.	No.	Date.	Monthly volume.		Official Gazette.	
			Spec.	Dr. g.	Vol.	Page.
General Electric Company.	(See Keller, Huldreich, assignor.)					
General Electric Company.	(See Kellogg, James W., assignor.)					
General Electric Company.	(See Key, James F., assignor.)					
General Electric Company.	(See Krohn, Sigvald, assignor.)					
General Electric Company.	(See Kruesi, August H., assignor.)					
General Electric Company.	(See La Cos, Joseph, assignor.)					
General Electric Company.	(See Larson, Carl W., assignor.)					
General Electric Company.	(See Latour, Marius C. A., assignor.)					
General Electric Company.	(See Lemp, Hermann, assignor.)					
General Electric Company.	(See Libby, Sam H., assignor.)					
General Electric Company.	(See Linn, John B., assignor.)					
General Electric Company.	(See Lord, Charles E., assignor.)					
General Electric Company.	(See Lundskog, Julius W., assignor.)					
General Electric Company.	(See Macloskie, George, assignor.)					
General Electric Company.	(See Marshall, Norman, assignor.)					
General Electric Company.	(See Merrill, Wilbur L., assignor.)					
General Electric Company.	(See Meyersberg, Gustav, assignor.)					
General Electric Company.	(See Milch, Maurice, assignor.)					
General Electric Company.	(See Miller, David, assignor.)					
General Electric Company.	(See Möller, Karl, assignor.)					
General Electric Company.	(See Moody, Walter S., assignor.)					
General Electric Company.	(See Morganthaler, Peter C., assignor.)					
General Electric Company.	(See Mortensen, Casper C. M., assignor.)					
General Electric Company.	(See Murphy, Edwin J., assignor.)					
General Electric Company.	(See Newton, Arthur D., assignor.)					
General Electric Company.	(See Noeggerath, Jakob E., assignor.)					
General Electric Company.	(See Ober, Julius E., assignor.)					

Figure 16: Book HJ1A7X (*Annual Report on Patents for the US House of Representatives, 1906*) containing the “paragraph” with the smallest bits-per-byte.

shown in Figure 16. The OCR engine parsed the column of “General Electric Company.” as a single block of text and the identified subtopic paragraph consists of several repetitions of the text “General Electric Company.” According to the QWEN3-0.6B-BASE model, this paragraph has BPB value 0.0297, reflecting its repetition.

The paragraphs identified with the largest BPB appear to consist mostly of OCR garbled text. These extreme examples confirm that BPB can be a helpful predictor of text quality and might be useful when filtering the data.

A total of 12,395 paragraphs ($\approx 0.0009\%$) carry the -1 sentinel in the released dataset. Of these, 12,391 are longer than the 14,134 token limit and 4 are too short (which are all OCR noise on the last page of books).

4.12 Final Annotation and Formatting

The final step of the pipeline is to collect the processed middlematter, the separated endmatter, the duplicate identification clusters, and the per-subtopic paragraph BPB values together to generate the enriched text output. In addition, we compute and incorporate some metadata.

Output Format. The pipeline emits three annotated strings per volume: `frontmatter_gen`, `middlematter_gen`, and `backmatter_gen`. Each is assembled from a small set of HTML-like tags and consists of HTML-escaped text. We choose HTML due to the availability of parsing tools and standards (though we also release a small, custom parser library, cf. Appendix E). HTML allows separation of annotation and primary content: this is the difference between HTML attributes/structural elements and inner text.

Inside the endmatter, each non-empty page becomes one `<div>`. There is exactly one `<div>` per endmatter page and empty pages are discarded. Each `<div>` is annotated by its detected class, taking the form

```
<div class="toc_index"> ... page text ... </div>
<div class="biblio"> ... page text ... </div>
<div class="otherendmatter"> ... page text ... </div>
```

The middlematter is grouped into subtopic sections, each of which consists of subtopic paragraphs. Each subtopic section is demarcated with a `<section>` tag. When available, each section tag carries a `data-bpb` annotation with the mean BPB of the section (computed as the non-weighted average BPB of the paragraphs in that section). This takes the form

```
<section data-bpb="X.XXXX"> ... paragraphs ... </section>
```

The BPB values are given to 4 decimal digits.

Each subtopic paragraph is indicated with a `<p>` tag and can carry up to four attributes:

- `data-bpb="X.XXXX"`: per-paragraph bits-per-byte to 4 decimal digits. This is omitted when the computed BPB is ≤ 0 (i.e. when it has the `-1` sentinel value).
- `data-language="LAN"`: per-paragraph detected language in ISO-639-3 from the POLYGLOT library. Can be UNKNOWN and is omitted if None.
- `data-representative`: a Boolean flag indicating that this paragraph is the chosen cluster id paragraph among a cluster of identified duplicate paragraphs (cf. §4.10). If present, this indicates that this paragraph has the volume with the alphabetically-first barcode among a class of duplicate paragraphs.
- `data-clusterid="BARCODE:PARIDX"`: when `data-representative` is present, this attribute will be present and gives the name of the cluster.

In addition, if a paragraph is identified as a duplicate of a *different* paragraph, then the paragraph is wrapped in an `aside` tag with a `data-cluster="BARCODE:PARINDICES"` attribute. If it is a single paragraph, the `data-cluster` takes the form `BARCODE:PARIDX`. But often a contiguous sequence of paragraphs duplicates another contiguous sequence of paragraphs. When this occurs, the `data-cluster` takes the form `BARCODE:N-M` where `N` and `M` are the first and last indices of the cluster source. Prototypical output resembles the following (with whitespace here emphasizing annotation structure):

```
<section data-bpb="X.XXXX">
  <p data-bpb="X.XXXX" data-language="LANG"
    data-representative data-clusterid="ABCDEF:15">
    ... source text for a duplicate class ...
  </p>
  <p data-bpb="X.XXXX" data-language="LANG">
    ... paragraph text ...
  </p>
  <aside data-cluster="GHIJKL:21">
    <p data-bpb="X.XXXX" data-language="LANG">
      ... duplicate text ...
    </p>
```

```

    </aside>
</section>
<section>
    ...
</section>

```

Stripping all HTML tags and parsing only HTML inner text recovers the volume text. However, the HTML attributes encode metadata that might be useful for filtering or structured reasoning.

The following 21 metadata descriptors are included:

bpb_min_gen	bpb_p10_gen	bpb_p30_gen
bpb_median_gen	bpb_p70_gen	bpb_p90_gen
bpb_max_gen	bpb_avg_gen	primary_language_gen
language_distribution_gen	token_count_gen	char_count_gen
word_count_gen	sentence_count_gen	paragraph_count_gen
section_count_gen	bigram_count_gen	bigram_count_unique_gen
trigram_count_gen	trigram_count_unique_gen	tokenizability_ratio_gen

We use the suffix “_gen” to indicate values computed by this pipeline, including all these metadata descriptors. The bpb_XXX_gen metadata contain the min, max, average, median, 10th percentile, 30th percentile, 70th percentile, and 90th percentile among the bits-per-byte values of subtopic paragraphs in that volume. The languages and proportions are encoded as

- `primary_language_gen`: the primary language code (given as ISO 639-3 names)¹⁹ as reported in IB-HL.
- `language_distribution_gen`: a dictionary of (lang: proportion) pairs, where lang is an ISO 639-3 language code and proportion is a float from 0.0 to 1.0 describing the proportion of the paragraphs in that language in the book.

The remaining metadata follows the same naming convention as in IB-HL (see §4.8 of Cargnelutti et al. 2025).

Finally, we include one filtered plaintext version of the collection called `processed_middlematter_gen`. This is a plaintext version that skips duplicate paragraphs (keeping only the representative cluster id for each class), keeps only paragraphs with paragraph-level BPB between the 10th and 90th volume-level percentile, and otherwise removes all other annotations. We provide this as an easy way to directly interact with a filtered version of the dataset. This serves as an opinionated baseline for users to compare against when parsing and filtering the dataset.

4.12.1 Methodology

All volume text is HTML-escaped using `html.escape(quote=False)` in Python during assembly. The remaining work consists of computing additional metadata and standard string formatting.

In IB-HL, effort was made to identify languages on chunks of text up to 768 characters long (cf. §4.4.1 of Cargnelutti et al. 2025). This provided information on the language distribution

¹⁹<https://www.iso.org/iso-639-language-code>

within each volume. In this pipeline, we run a language-detection algorithm on every subtopic paragraph. As subtopic paragraphs correspond more closely to semantic chunks, this should give a more granular view of language distribution.

We use the POLYGLOT²⁰ library (Al-Rfou, Perozzi, and Skiena 2013) to detect the language of each subtopic paragraph with one minor modification: if POLYGLOT fails to detect the language for a particular paragraph, and that paragraph has fewer than 30 o200K_BASE tokens (computed using (OpenAI 2026)), *and* the two surrounding paragraphs had positive language detections with the same language, then we assume the middle paragraph also has that same language. Language detection accuracy degrades on short texts; however, contiguous chunks are typically in the same language.

For the rest of the metadata: token count and tokenizability are computed using TIKTOKEN (OpenAI 2026) with respect to the o200K_BASE tokenizer. “Tokenizability” is a score that measures how efficiently o200K_BASE can encode the text; in particular, it measures how close to 1.25 tokens per word the text is. Word counts and lists are computed using POLYGLOT. Sentence counts, paragraph counts, and section counts are computed using the segmentation and chunking from above. Bigrams and trigrams are computed in pure Python over the POLYGLOT word lists.

4.12.2 Results

Metadata computation required approximately 150h (combined wall-time on a 16-core MacBook Pro M4 Max). Exact timing was not logged. Table 17 contains detected language counts. This table contains the top ten most commonly detected languages in the collection and the number of subtopic paragraphs detected in that language. For comparison, we also present the number of volumes having that language as the dominant language according to IB-HL.

Language	Detected Subtopic Pars in Language	(% of All Pars)	Volumes with this Primary Language	(% of All Volumes)
English	732,171,247	52.52%	487,353	49.58%
German	221,319,743	15.88%	157,776	16.05%
French	174,288,052	12.50%	135,872	13.82%
Latin	51,553,325	3.70%	21,640	2.20%
Italian	42,287,689	3.03%	46,074	4.69%
Spanish	23,468,303	1.68%	28,836	2.93%
Russian	21,807,544	1.56%	15,115	1.54%
Dutch	15,615,935	1.12%	12,682	1.29%
Greek	13,726,684	0.98%	6,319	0.64%
Danish	13,142,232	0.94%	7,899	0.80%

Table 17: Paragraph Language Data

The per-paragraph distribution closely agrees with the per-book distribution for large languages. We observe that Latin (lat) has only 2.20% of books in IB-HL and 3.70% of paragraphs across the collection. Many non-Latin books include quotes or passages in Latin.

As partial validation, we compared the detected primary language from IB-HL against the most common language (by paragraph count) detected in each volume in this pipeline. We found 98% agreement, and volumes with differing identified languages tended to be either

²⁰<https://github.com/aboSamoor/polyglot>

bilingual or in pairs of languages with large overlaps in n-grams, such as German-Yiddish-Dutch or Occitan-Catalan or Montenegrin-Croatian.

5 Dataset Analysis

We present three focused analyses on separate aspects of the pipeline. Each analysis serves to validate part of the pipeline and to illustrate research directions facilitated by IB-HL-ET. First, we compare text statistics of IB-HL-ET against IB-HL to quantify the effect of cleaning. Second, we combine duplicate identification with language detection to measure reuse *across* languages. Third, we pair BPB with publication metadata to trace how text predictability changes across the Harvard Library book collection. Each of these analyses depends on a different annotation layer added in this pipeline.

5.1 Tokenizability in IB-HL and IB-HL-ET

Constructing IB-HL-ET extends the text-refinement process begun in IB-HL. We quantify the differences between this dataset and IB-HL by comparing statistics. Table 18 contains character, bigram, trigram, and tokenizability statistics. (The rows for IB-HL-ET correspond to the final middlematter with all annotations removed). The IB-HL-ET dataset has ≈ 26 B fewer characters than IB-HL; this is primarily due to separating endmatter, though running header/footer removal and page number removal have a small contribution. The bigram and trigram unique ratios are smaller by at least 5 percentage points, while tokenizability increases by ≈ 6 points. These are all consistent with less noise and effective cleaning.

Dataset	# Characters	Bigram uniq %	Trigram uniq %	Tokenizability
IB-HL (src OCR)	828.48B	46.40%	74.46%	80.43
IB-HL-ET	802.11B	38.79%	69.23%	86.57
IB-HL (Top-5 Langs)	729.65B	37.79%	68.45%	88.63
IB-HL-ET (Top-5 Langs)	709.90B	37.24%	68.19%	89.24

Table 18: Character and n-gram statistics between IB-HL and this dataset (IB-HL-ET). The *Top-5 Langs* are separated because IB-HL performed additional text analysis and OCR cleaning on these languages.

Lang	Volumes	Tokenizability			Chars (B)		Bigram uniq %	
		Raw	IB-HL	IB-HL-ET	IB-HL	IB-HL-ET	IB-HL	IB-HL-ET
English	487,342	90.05	97.02	97.65	415.4	402.6	33.66	33.02
German	157,754	71.41	75.95	76.44	147.2	143.8	46.80	46.49
French	135,871	75.15	79.68	80.34	115.3	112.8	39.86	39.30
Italian	46,074	71.16	74.59	75.15	32.1	31.5	47.25	46.81
Spanish	28,836	76.88	80.89	81.69	19.6	19.2	39.45	38.62

Table 19: Per-language comparison on the 855,877 volumes with IB-HL post-processing. Excludes 34 volumes without post-processing in IB-HL.

IB-HL applied additional processing on books in the top 5 languages.²¹ The second set of rows in the table restricts to volumes with additional processing in IB-HL and shows small decreases

²¹IB-HL omitted 34 books having one of the 5 most common primary languages from additional processing. This causes some small differences in tables in this section.

Group	Volumes	IB-HL Tokenizability	IB-HL-ET Tokenizability
Top-5 Langs	855,911	88.63	89.24
Not Top-5 Langs	127,091	64.56	68.53
NUPUNKT Langs	962,373	85.85	86.82
(NUPUNKT Langs Excl. Top-5 Langs)	106,462	63.53	67.33
SAT Langs	20,629	69.84	74.72

Table 20: Per-group comparison of tokenizability between IB-HL and IB-HL-ET.

in bigram and trigram unique ratios and a small increase in tokenizability. Table 19 contains a per-language breakdown for the top 5 languages and demonstrates small, but consistent, differences. Table 20 shows that the largest difference in tokenizability comes from volumes in SAT languages.

Analysis shows that this pipeline increased tokenizability for every language. The largest increases are Occitan (+9.5), Persian (+5.8), and Arabic (+5.6).

5.2 Multilingual Reuse of Duplicated Text

Corpus-wide duplicate identification and language detection allow tracking how often a passage in one language occurs *outside* books in its language. For every duplicated subtopic paragraph, we compare its detected language against the primary language of the volume that contains it. Across all 124,049,981 duplicated paragraphs, only 5.3% have a detected language different from their host volume.

Paragraph language	Dup. pars (≥200 ch)	Own-lang % (all lengths)	Own-lang % (≥200 ch)	Top host languages (≥200 ch)
English	76,448,359	98	99	German, French (<1% each)
German	7,240,400	94	97	English 2%, French 1%
Russian	144,414	96	97	German 1%, Bulgarian 1%
French	7,862,913	93	95	German 2%, English 2%
Dutch	275,237	81	93	German 2%, French 2%
Spanish	519,462	88	92	English 6%, French 1%
Italian	812,897	85	91	German 4%, English 3%
Danish	152,055	44	87	English 4%, German 3%
Portuguese	87,953	57	82	English 7%, Spanish 6%
Latin	1,466,807	53	61	English 15%, German 10%
Greek	82,451	51	55	English 25%, German 10%

Table 21: How often duplicated paragraphs appear outside volumes of their own language. For each detected paragraph language we give the share of its duplicated paragraphs hosted in a same-language volume, computed over all duplicated paragraphs and over only those of at least 200 characters. Host-language shares and counts are for the ≥200-character subset.

Some of this cross-lingual sharing is an artifact of language detection on short fragments. Roman-numeral lists, headers of tables, marginalia-based citations, and other commonly mis-OCR'd text confuse our language detection algorithms. Paragraphs of at least 200 characters are predominantly prose, and the added length gives enough context for more accurate language determination. In Table 21, we describe the 2.3% of paragraphs having at least 200

characters and that have a detected language that differs from their host volume.

The largest languages in IB-HL-ET are strongly same-language-bound: 95–99% of duplicated English, German, French, and Russian (≥ 200 ch) paragraphs occur in volumes whose primary language matches. Classical and scholarly languages behave differently. Latin and Greek paragraphs often appear in texts with different primary languages. This distribution reveals the collecting practices of Harvard Library as much as it reveals trends in reusing text. The reuse patterns are consistent with text circulation: legal, classical, and liturgical passages are quoted verbatim across scholarly literature written in many languages.

The difference between the distributions among all paragraphs and ≥ 200 ch paragraphs shows biases within our language detection algorithm. Disproportionately many small subtopic paragraphs are recognized as Danish or Portuguese. The relative consistency of Latin and Greek across the two distributions suggests true reproduction and quotation instead of noise.

5.3 Bits-Per-Byte Over Time

IB-HL compiled publication years for every volume. In this pipeline, we computed the BPB of every subtopic paragraph. Combining these two distributions allows comparing BPB across centuries of printing (see Figure 22). The dominant feature is a gradual decline: text becomes more predictable to QWEN3-0.6B-BASE as it approaches the present. Median per-volume BPB falls from ≈ 2.5 in the 17th century to ≈ 1.4 in the mid-19th century and ≈ 1.0 by the mid-20th century.

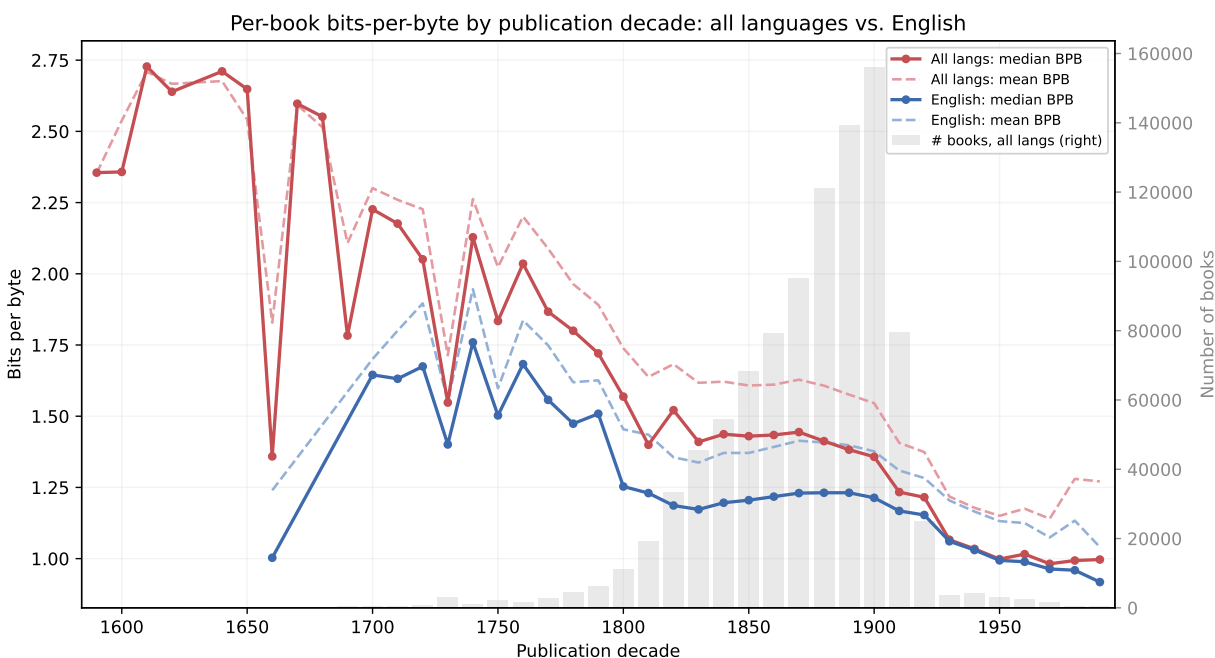


Figure 22: Per-book bits-per-byte by publication decade, all languages (red) versus English (blue). Solid lines are the median of per-book median BPB; dashed lines the mean of per-book mean BPB. Grey bars give the number of dated books per decade (right axis). Decades with fewer than 50 books are omitted.

We compare English (blue) against all languages (red). The English subset traces approxi-

mately the same shape²² but with a slightly smaller median.

One interpretation of this trend is that BPB measures surprise under a modern language model, hence this curve approximates *distance to contemporary prose*. Older orthography, vocabulary, typesetting, and styles are less predictable to modern language models. The larger amount of available text from the 19th and 20th centuries (corresponding to the increased number of available volumes from that era) might appear in model training data, consistent with the observed decrease in BPB.

We note two potential confounding variables: OCR noise and sample composition. Due to physical degradation and different publishing norms, older volumes have less consistent OCR. BPB conflates model surprise with OCR noise, hence older volumes likely have higher BPB purely for OCR reasons.²³ In addition, the number of available volumes drops sharply at the public-domain boundary in the 1930s. The available volumes from after this cutoff are not representative of all that was printed. Continued BPB decline after 1930 may be partially explained by selectivity bias among volumes that already appear in the public domain.

6 Discussion and Further Directions

Like the dataset on which it is based, IB-HL-ET is a step in an ongoing, collaborative process rather than a finished artifact. By choosing to *annotate* rather than *remove* paratextual elements and metadata, we hope this dataset will be a useful starting point that others can easily tailor to their needs. The philosophy of annotation over removal invites extension: as additional use cases and needs arise, we may identify additional annotations or analyses to include. In time, we hope this methodology and the resulting data format will prove useful to others in creating datasets from cultural texts. We believe that the adoption of a standardized format for enriching text has the potential to make data practices in AI model training more robust while improving the overall computational addressability and cross-compatibility of text corpora.

Analysis of the Enriched Text. Beyond serving as an input to model training, the enriched dataset opens lines of research that a flattened token stream would not. Earlier analyses (cf. §5) illustrated a few. Because duplicate paragraphs are marked rather than deleted (§4.10), the collection is a map of textual reuse: Table 21 and §5.2 reveal a rich, cross-lingual structure of quotation and clustering. Pairing per-paragraph BPB with publication year (Figure 22, §5.3) demonstrates that the corpus enables study of historical language change, raising questions about per-language writing trends, era-specific OCR degradation, and training data availability. We hope that IB-HL-ET enables many research directions, including directions we have not anticipated.

Pipeline Improvements. Limitations in recognizing tables, marginalia, and technical language lead to weaknesses in the current dataset. Examples of two known weaknesses are

²²The 1660s have an unusually low BPB. This is due to 58 volumes of *Acts and Resolves passed by the General Court* (the colonial Massachusetts Bay legislative record). These have high OCR quality, have formulaic statutory prose, and have correspondingly low median BPB. The average BPB among the 21 other English volumes from that decade is ≈ 1.64 .

²³ On the other hand, OCR confidence metadata indicates that English OCR is relatively consistent, and hence the English curve suggests a clear trend and not merely OCR noise. Disentangling these variations requires additional research.

shown in Appendix D. We expect advances in OCR will address each of these, in turn improving the pipeline.

This pipeline was built against the OCR provided for Harvard Library’s collection. It is possible that different collections and different OCR pipelines may reveal different failure modes and pose new engineering challenges.

Towards a Virtuous Cycle. We are working to establish diverse collaborations on this and related datasets. We want this work, together with IB-HL, to make millions more books computationally accessible. We also hope that improved training data for low-resource languages will lead to improved models, allowing more accurate analysis across the collection. As noted by Beyene and Dancy 2026, there is a growing body of work demonstrating technical capacity to process historical documents, but a lack of prioritization in training data and evaluation benchmarks. We release the pipeline and data with the goal that they are extended and applied to further collections. As communities extend the annotations, develop new cleaning techniques, correct errors, and add perspectives, those improvements can be incorporated into the source material. Crucially, that cycle must preserve provenance, reproducibility, and the ability to audit transformations.

7 Acknowledgements

We thank Harvard Library for their support and for giving us the opportunity to continue working on this unique collection. Many computations in this paper were run on the FASRC Cannon cluster supported by the FAS Division of Science Research Computing Group at Harvard University. We thank Kacie Bailey, Matte Hartog, and Jimmy Mendez for their support and encouragement.

We also thank Ted Underwood, Nick Levine, and Alec Radford for helpful comments and suggestions, especially regarding annotation, normalization, and bits-per-byte filtering. Incorporating this feedback has led to an improved pipeline, dataset, and report.

The pipeline uses many libraries and projects mentioned in this report. We also credit technical libraries that we used for exploration, data analysis, prototyping, distributing work across compute nodes, and preparing this technical report. We used JUPYTER (Kluyver et al. 2016), PANDAS (McKinney 2010; The Pandas Development Team 2020), Apache ARROW, and SCIKIT-LEARN (Pedregosa et al. 2011; Buitinck et al. 2013) in essential ways at many stages of exploration and development. We used PYBIND11 (Jakob, Rhineland, and Moldovan 2024) to write Python extensions in C++ for performance, orchestrated tests using PYTEST (Krekel et al. 2004) to ensure stability across heterogeneous compute environments, distributed runs using GNU PARALLEL (Tange 2011), and monitored progress using TQDM (Costa-Luis et al. 2026) bars and LOGURU logging. Finally, we used NUMPY (Harris et al. 2020) and MATPLOTLIB (Hunter 2007) to generate all figures in this report.

This work was supported by unrestricted funding from Microsoft, OpenAI, Meta, and Jane Street.

AI tools were used to assist in the preparation of this technical report.

8 Rights Determination

We respect the intellectual property rights of authors, publishers, and other rights holders. While we have taken deliberate steps to include only those volumes for which there is no known copyright restriction, specifically those identified by the HathiTrust Digital Library with a status of “public domain,” “public domain in the United States,” or “CC-Zero,” copyright determinations are complex and context-dependent, and occasionally subject to error.

While this is relatively low risk, some volumes in this dataset may be in the public domain in the United States but still subject to copyright or other rights protections in other jurisdictions. Additionally, the absence of an explicit copyright claim or rights status does not guarantee that a work is in the public domain, either in the U.S. or abroad. Information about the copyright status of individual volumes is provided on a good-faith basis and reflects available data at the time of determination, but we cannot guarantee its completeness or accuracy.

Users of this dataset will be solely responsible for making independent legal assessments about how and where they use the materials. Some uses of materials may also be restricted by trademark, privacy, publicity rights, or other such rights or restrictions. It is the user’s sole responsibility to consider the possibility that such rights or restrictions may be involved and to secure any needed permissions. If any rights holder believes that a work included in this release is misidentified or improperly included, we welcome contact and will promptly review any concerns. Our goal is to provide broad public access while maintaining respect for intellectual property rights and ensuring responsible data stewardship.

9 Disclaimers

9.1 Harmful Language and Content in this Dataset

This dataset is a collection of historical works that reflect the language, imagery, culture, and perspectives of their time. Users should be aware that some materials may contain language or portrayals that are outdated, offensive, or harmful today, such as racism, sexism, colonial attitudes, and other forms of discrimination. Some content may include inaccurate information, providing insight into historical contexts that existed at the time of writing. The materials are maintained in their original form to retain contextual understanding and facilitate research efforts, but we encourage critical awareness and cultural sensitivity for the creators and/or subjects of the collection. These materials are offered as part of a historical perspective, but should not be considered a stand-alone research collection constructed to give a balanced perspective on any topic.

9.2 Harmful Language in Bibliographic Description

Metadata for this collection may contain language that is overtly or implicitly harmful, outdated, or biased, or may by omission fail to represent important perspectives. Metadata may contain language created decades ago. It is common practice within the field of library science to reuse descriptions provided from the creator of the materials. While in some instances this allows communities and individuals to represent their materials in their own words, unexamined use of this practice may mean that racist or other offensive terminologies appear in our description. We also use national standardized terms in our work that can be outdated and harmful. Note that terminology in historical materials and in library descriptions does

not always match the language we currently understand to be preferred by members of the communities depicted.

Furthermore, we acknowledge that the act of collecting materials is not always neutral, and the work of describing and classifying library materials is influenced by inherent personal, institutional, and societal biases. Outdated or offensive terminologies may be present in meta-data such as subject headings, and harmful language or bias may be introduced by catalogers supplying titles and descriptions. In other cases, books themselves present racist, offensive or otherwise harmful viewpoints in titles or descriptions that are routinely transcribed by catalogers.

Note: Some language in this statement was adopted from Harvard Library’s statement on Harmful Language in Library collections.²⁴

9.3 Generated and Experimental Content

This dataset contains generated and/or experimental content. While reasonable care was taken to ensure its quality, it is provided “as is,” without warranties of any kind. It may contain errors or inaccuracies; users should verify the data independently and apply their own judgement.

Reference list

Abbas, A., K. Tirumala, D. Simig, S. Ganguli, and A. S. Morcos (2023). *SemDeDup: Data-efficient learning at web-scale through semantic deduplication*. [online]. doi: [10.48550/arXiv.2303.09540](https://doi.org/10.48550/arXiv.2303.09540). arXiv Preprint: [2303.09540](https://arxiv.org/abs/2303.09540) (cs.LG) (cit. on p. 20).

Allal, L. B., A. Lozhkov, and E. Bakouch (July 2024). *SmolLM - blazingly fast and remarkably powerful*. [online]. Accessed July 2026. URL: <https://huggingface.co/blog/smollm> (cit. on p. 10).

Al-Rfou, R., B. Perozzi, and S. Skiena (2013). “Polyglot: Distributed word representations for multilingual NLP.” *Proceedings of the Seventeenth Conference on Computational Natural Language Learning*. POLYGLOT code at <https://github.com/aboSamoor/polyglot>, pp. 183–192 (cit. on p. 29).

Ansel, J., E. Yang, H. He, N. Gimpel, A. Jain, M. Voznesensky, B. Bao, P. Bell, D. Berard, E. Burovski, G. Chauhan, A. Chourdia, W. Constable, A. Desmaison, Z. DeVito, E. Ellison, W. Feng, J. Gong, M. Gschwind, B. Hirsh, S. Huang, K. Kalambarkar, L. Kirsch, M. Lazos, M. Lezcano, Y. Liang, J. Liang, Y. Lu, C. Luk, B. Maher, Y. Pan, C. Puhersch, M. Reso, M. Saroufim, M. Y. Siraichi, H. Suk, M. Suo, P. Tillet, E. Wang, X. Wang, W. Wen, S. Zhang, X. Zhao, K. Zhou, R. Zou, A. Mathews, G. Chanan, P. Wu, and S. Chintala (Apr. 2024). “PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation.” *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS ’24)*. ACM. doi: [10.1145/3620665.3640366](https://doi.org/10.1145/3620665.3640366). URL: <https://docs.pytorch.org/assets/pytorch2-2.pdf> (cit. on p. 54).

Appleby, A. (2016). *SMHasher*. [GitHub Code Repository]. URL: <https://github.com/aappleby/smhasher> (cit. on pp. 9, 47).

Beyene, F. S. and C. L. Dancy (2026). *A Survey of OCR Evaluation Methods and Metrics and the Invisibility of Historical Documents*. [online]. doi: [10.48550/arXiv.2603.25761](https://doi.org/10.48550/arXiv.2603.25761). arXiv Preprint: [2603.25761](https://arxiv.org/abs/2603.25761) (cs.CV) (cit. on pp. 8, 34).

²⁴<https://library.harvard.edu/harmful-language-library-collections>

- Bommarito, M. J., D. M. Katz, and J. Bommarito (2025). *Precise Legal Sentence Boundary Detection for Retrieval at Scale: NUPunkt and CharBoundary*. [online]. NUPUNKT code at <https://github.com/alea-institute/nupunkt>. doi: [10.48550/arXiv.2504.04131](https://doi.org/10.48550/arXiv.2504.04131). arXiv Preprint: [2504.04131](https://arxiv.org/abs/2504.04131) (cs.CL) (cit. on pp. 6, 16).
- Brants, T., A. Popat, P. Xu, F. J. Och, and J. Dean (2007). “Large language models in machine translation.” *Proceedings of the 2007 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning (EMNLP-CoNLL)*, pp. 858–867 (cit. on pp. 13, 48).
- Broder, A. Z. (2000). “Identifying and filtering near-duplicate documents.” *Annual Symposium on Combinatorial Pattern Matching*. Springer, pp. 1–10 (cit. on p. 14).
- Brown, T., B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al. (2020). “Language models are few-shot learners.” *Advances in Neural Information Processing Systems* 33, pp. 1877–1901 (cit. on p. 20).
- Buitinck, L., G. Louppe, M. Blondel, F. Pedregosa, A. Mueller, O. Grisel, V. Niculae, P. Prettenhofer, A. Gramfort, J. Grobler, R. Layton, J. VanderPlas, A. Joly, B. Holt, and G. Varoquaux (2013). “API design for machine learning software: experiences from the scikit-learn project.” *ECML PKDD Workshop: Languages for Data Mining and Machine Learning*, pp. 108–122 (cit. on p. 34).
- Cargnelutti, M., C. Brobston, J. Hess, J. Cushman, K. Mukk, A. Scourtas, K. Courtney, G. Leppert, A. Watson, M. Whitehead, et al. (2025). *Institutional Books 1.0: A 242B token dataset from Harvard Library’s collections, refined for accuracy and usability*. [online]. doi: [10.48550/arXiv.2506.08300](https://doi.org/10.48550/arXiv.2506.08300). arXiv Preprint: [2506.08300](https://arxiv.org/abs/2506.08300) (cs.CL) (cit. on pp. 3, 10, 28).
- Charikar, M. S. (2002). “Similarity estimation techniques from rounding algorithms.” *Proceedings of the thirty-fourth Annual ACM Symposium on Theory of Computing*, pp. 380–388 (cit. on pp. 9, 20).
- Chen, J., S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu (2024). *BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation*. [online]. doi: [10.48550/arXiv.2402.03216](https://doi.org/10.48550/arXiv.2402.03216). arXiv Preprint: [2402.03216](https://arxiv.org/abs/2402.03216) (cs.CL) (cit. on pp. 7, 48).
- Chiron, G., A. Doucet, M. Coustaty, and J.-P. Moreux (2017). “ICDAR2017 competition on post-OCR text correction.” *2017 14th IAPR International Conference on Document Analysis and Recognition (ICDAR)*. Vol. 1. IEEE, pp. 1423–1428 (cit. on p. 45).
- Choi, F. Y. (2000). “Advances in domain independent linear text segmentation.” *1st Meeting of the North American Chapter of the Association for Computational Linguistics* (cit. on p. 53).
- Cormen, T. H., C. E. Leiserson, R. L. Rivest, and C. Stein (2009). *Introduction to Algorithms, 3rd Ed.* MIT Press (cit. on p. 9).
- Costa-Luis, C. da, S. K. Larroque, K. Altendorf, H. Mary, richardsheridan, M. Korobov, N. Raphael, I. Ivanov, M. Bargull, N. Rodrigues, Shawn, M. Dektiarev, M. Górný, mjstevens777, M. D. Pagel, M. Zugnoni, CrazyPython, C. Newey, A. Lee, pgajdos, Todd, S. Malmgren, redbug312, O. Desh, N. Nechaev, M. Boyle, M. Nordlund, MapleCCC, J. McCracken, and H. van Kemenade (Feb. 2026). *tqdm: A fast, Extensible Progress Bar for Python and CLI*. Version v4.67.3. doi: [10.5281/zenodo.18473238](https://doi.org/10.5281/zenodo.18473238) (cit. on p. 34).
- Devlin, J., M.-W. Chang, K. Lee, and K. Toutanova (2019). “BERT: Pre-training of deep bidirectional transformers for language understanding.” *Proceedings of the 2019 conference of the North American*

chapter of the *Association for Computational Linguistics: Human Language Technologies, volume 1 (long and short papers)*, pp. 4171–4186 (cit. on p. 45).

Dodge, J., M. Sap, A. Marasović, W. Agnew, G. Ilharco, D. Groeneveld, M. Mitchell, and M. Gardner (2021). “Documenting large webtext corpora: A case study on the colossal clean crawled corpus.” *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 1286–1305 (cit. on p. 45).

Frohmman, M., I. Sterner, I. Vulić, B. Minixhofer, and M. Schedl (Nov. 2024). “Segment Any Text: A Universal Approach for Robust, Efficient and Adaptable Sentence Segmentation.” *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Ed. by Y. Al-Onaizan, M. Bansal, and Y.-N. Chen. Miami, Florida, USA: Association for Computational Linguistics, pp. 11908–11941. doi: [10.18653/v1/2024.emnlp-main.665](https://doi.org/10.18653/v1/2024.emnlp-main.665) (cit. on p. 16).

Gao, L., S. Biderman, S. Black, L. Golding, T. Hoppe, C. Foster, J. Phang, H. He, A. Thite, N. Nabeshima, et al. (2020). *The Pile: An 800gb dataset of diverse text for language modeling*. [online]. doi: [10.48550/arXiv.2101.00027](https://doi.org/10.48550/arXiv.2101.00027). arXiv Preprint: [2101.00027](https://arxiv.org/abs/2101.00027) (cs.CL) (cit. on pp. 20, 24, 45).

Gemma Team et al. (2025). *Gemma 3 Technical Report*. [online]. doi: [10.48550/arXiv.2503.19786](https://doi.org/10.48550/arXiv.2503.19786). arXiv Preprint: [2503.19786](https://arxiv.org/abs/2503.19786) (cs.CL) (cit. on pp. 10, 45).

Genette, G. and M. Maclean (1991). “Introduction to the Paratext.” *New Literary History* 22.2, pp. 261–272. ISSN: 00286087, 1080661X. doi: [10.2307/469037](https://doi.org/10.2307/469037) (cit. on p. 3).

GLM-5-Team et al. (2026). *GLM-5: from Vibe Coding to Agentic Engineering*. [online]. doi: [10.48550/arXiv.2602.15763](https://doi.org/10.48550/arXiv.2602.15763). arXiv Preprint: [2602.15763](https://arxiv.org/abs/2602.15763) (cs.LG) (cit. on p. 45).

Gunasekar, S., Y. Zhang, J. Aneja, C. C. T. Mendes, A. Del Giorno, S. Gopi, M. Javaheripi, P. Kauffmann, G. de Rosa, O. Saarikivi, et al. (2023). *Textbooks are all you need*. [online]. doi: [10.48550/arXiv.2306.11644](https://doi.org/10.48550/arXiv.2306.11644). arXiv Preprint: [2306.11644](https://arxiv.org/abs/2306.11644) (cs.CL) (cit. on pp. 3, 10).

Harris, C. R., K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. F. del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, and T. E. Oliphant (Sept. 2020). “Array programming with NumPy.” *Nature* 585.7825, pp. 357–362. doi: [10.1038/s41586-020-2649-2](https://doi.org/10.1038/s41586-020-2649-2) (cit. on p. 34).

Heafield, K. (July 2011). “KenLM: Faster and Smaller Language Model Queries.” *Proceedings of the Sixth Workshop on Statistical Machine Translation*. Ed. by C. Callison-Burch, P. Koehn, C. Monz, and O. F. Zaidan. Edinburgh, Scotland: Association for Computational Linguistics, pp. 187–197. URL: <https://aclanthology.org/W11-2123/> (cit. on pp. 48, 54).

Hearst, M. A. (1997). “Text tiling: Segmenting text into multi-paragraph subtopic passages.” *Computational Linguistics* 23.1, pp. 33–64 (cit. on pp. 18, 53).

Hsu, P.-L., Y. Dai, V. Kothapalli, Q. Song, S. Tang, S. Zhu, S. Shimizu, S. Sahni, H. Ning, and Y. Chen (2024). *Liger Kernel: Efficient Triton Kernels for LLM training*. [online]. doi: [10.48550/arXiv.2410.10989](https://doi.org/10.48550/arXiv.2410.10989). arXiv Preprint: [2410.10989](https://arxiv.org/abs/2410.10989) (cs.LG) (cit. on p. 56).

Hunter, J. D. (2007). “Matplotlib: A 2D graphics environment.” *Computing in Science & Engineering* 9.3, pp. 90–95. doi: [10.1109/MCSE.2007.55](https://doi.org/10.1109/MCSE.2007.55) (cit. on p. 34).

ICU (2020). *ICU Normalization Documentation*. <https://unicode-org.github.io/icu/userguide/transforms/normalization/> (cit. on p. 8).

Jakob, W., J. Rhineland, and D. Moldovan (2024). *pybind11 - Seamless operability between C++11 and Python*. [GitHub Code Repository]. URL: <https://github.com/pybind/pybind11> (cit. on p. 34).

Jatowt, A., M. Coustaty, N.-V. Nguyen, A. Doucet, et al. (2019). “Deep statistical analysis of OCR errors for effective post-OCR processing.” *2019 ACM/IEEE Joint Conference on Digital Libraries (JCDL)*. IEEE, pp. 29–38. doi: [10.1109/JCDL.2019.00015](https://doi.org/10.1109/JCDL.2019.00015) (cit. on p. 45).

Jiang, A. Q., A. Sablayrolles, A. Mensch, C. Bamford, D. Singh Chaplot, D. de las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, L. l. Renard Lavaud, M.-A. Lachaux, P. Stock, T. Le Scao, T. Lavril, T. Wang, T. Lacroix, and W. El Sayed (Oct. 2023). *Mistral 7B*. [online]. doi: [10.48550/arXiv.2310.06825](https://doi.org/10.48550/arXiv.2310.06825). arXiv Preprint: [2310.06825](https://arxiv.org/abs/2310.06825) (cs.CL) (cit. on p. 45).

Jurafsky, D. and J. H. Martin (2026). *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition, with Language Models*. 3rd. Online manuscript released January 6, 2026. URL: <https://web.stanford.edu/~jurafsky/slp3/> (cit. on p. 12).

Kaplan, J., S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei (2020). *Scaling laws for neural language models*. [online]. doi: [10.48550/arXiv.2001.08361](https://doi.org/10.48550/arXiv.2001.08361). arXiv Preprint: [2001.08361](https://arxiv.org/abs/2001.08361) (cs.LG) (cit. on p. 3).

Katz, S. (1987). “Estimation of probabilities from sparse data for the language model component of a speech recognizer.” *IEEE Transactions on Acoustics, Speech, and Signal Processing* 35.3, pp. 400–401 (cit. on p. 48).

Kimi Team et al. (2025). *Kimi K2: Open Agentic Intelligence*. [online]. doi: [10.48550/arXiv.2507.20534](https://doi.org/10.48550/arXiv.2507.20534). arXiv Preprint: [2507.20534](https://arxiv.org/abs/2507.20534) (cs.LG) (cit. on p. 45).

Kiss, T. and J. Strunk (2006). “Unsupervised multilingual sentence boundary detection.” *Computational Linguistics* 32.4, pp. 485–525 (cit. on p. 16).

Kluyver, T., B. Ragan-Kelley, F. Pérez, B. Granger, M. Bussonnier, J. Frederic, K. Kelley, J. Hamrick, J. Grout, S. Corlay, P. Ivanov, D. Avila, S. Abdalla, C. Willing, and J. development team (2016). “Jupyter Notebooks - a publishing format for reproducible computational workflows.” *Positioning and Power in Academic Publishing: Players, Agents and Agendas*. Ed. by F. Loizides and B. Schmidt. Netherlands: IOS Press, pp. 87–90. URL: <https://eprints.soton.ac.uk/403913/> (cit. on p. 34).

Kohlschütter, C., P. Fankhauser, and W. Nejdl (2010). “Boilerplate detection using shallow text features.” *Proceedings of the third ACM International Conference on Web Search and Data Mining*, pp. 441–450 (cit. on p. 45).

Krekel, H., B. Oliveira, R. Pfannschmidt, F. Bruynooghe, B. Laughner, and F. Bruhin (2004). *pytest 9.0.2*. <https://github.com/pytest-dev/pytest>. [GitHub Code Repository] (cit. on p. 34).

Kudo, T. and J. Richardson (Nov. 2018). “SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing.” *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Ed. by E. Blanco and W. Lu. Brussels, Belgium: Association for Computational Linguistics, pp. 66–71. doi: [10.18653/v1/D18-2012](https://doi.org/10.18653/v1/D18-2012) (cit. on p. 54).

Kukich, K. (1992). “Techniques for automatically correcting words in text.” *ACM Computing Surveys (CSUR)* 24.4, pp. 377–439 (cit. on p. 45).

Lee, J. K. (2014). “The Korean Punctuation Systems.” *Acta Linguistica Asiatica* 4.1, pp. 29–41 (cit. on p. 17).

Lee, K., D. Ippolito, A. Nystrom, C. Zhang, D. Eck, C. Callison-Burch, and N. Carlini (2022). “Deduplicating training data makes language models better.” *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 8424–8445 (cit. on p. 20).

Leskovec, J., A. Rajaraman, and J. D. Ullman (2020). *Mining of Massive Datasets*. Cambridge University Press (cit. on p. 14).

Levine, N., D. Duvenaud, and A. Radford (Apr. 2026). *Introducing talkie: a 13B vintage language model from 1930*. [online]. URL: <https://talkie-lm.com/introducing-talkie> (cit. on p. 3).

Lhoest, Q., A. Villanova del Moral, Y. Jernite, A. Thakur, P. von Platen, S. Patil, J. Chaumond, M. Drame, J. Plu, L. Tunstall, J. Davison, M. Šaško, G. Chhablani, B. Malik, S. Brandeis, T. Le Scao, V. Sanh, C. Xu, N. Patry, A. McMillan-Major, P. Schmid, S. Gugger, C. Delangue, T. Matussière, L. Debut, S. Bekman, P. Cistac, T. Goehringer, V. Mustar, F. Lagunas, A. Rush, and T. Wolf (Nov. 2021). “Datasets: A Community Library for Natural Language Processing.” *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Library available at <https://github.com/huggingface/datasets>. Association for Computational Linguistics, pp. 175–184. doi: 10.18653/v1/2021.emnlp-demo.21. arXiv Preprint: 2109.02846 (cs.CL) (cit. on p. 58).

Li, J., A. Fang, G. Smyrnis, M. Ivgi, M. Jordan, S. Gadre, H. Bansal, E. Guha, S. Keh, K. Arora, et al. (2024). “Datacomp-LM: In search of the next generation of training sets for language models.” *Advances in Neural Information Processing Systems* 37, pp. 14200–14282 (cit. on p. 45).

Li, Y., R. Bonatti, S. Abdali, J. Wagle, and K. Koishida (2024). “Data generation using large language models for text classification: An empirical case study.” [online]. doi: 10.48550/arXiv.2407.12813. arXiv Preprint: 2407.12813 (cs.CL) (cit. on p. 10).

Li, Z., H. Zhu, Z. Lu, and M. Yin (2023). “Synthetic data generation with large language models for text classification: Potential and limitations.” *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 10443–10461 (cit. on p. 10).

Liu, Z., L. Tang, L. Jin, H. Li, N. Ranjan, D. Fan, S. Rohatgi, R. Fan, O. Pangarkar, H. Wang, et al. (2025). *K2-V2: A 360-Open, Reasoning-Enhanced LLM*. [online]. doi: 10.48550/arXiv.2512.06201. arXiv Preprint: 2512.06201 (cs.LG) (cit. on p. 3).

Luo, X., Z. Shinnick, N. Griesshaber, Y. Wang, J. Yu, F. Shi, P. Torr, and Y. Lu (2026). *Pretraining Language Models on Historical Text*. [online]. doi: 10.48550/arXiv.2606.02991. arXiv Preprint: 2606.02991 (cs.CL) (cit. on p. 3).

Marion, M., A. Üstün, L. Pozzobon, A. Wang, M. Fadaee, and S. Hooker (2023). *When Less is More: Investigating Data Pruning for Pretraining LLMs at Scale*. [online]. doi: 10.48550/arXiv.2309.04564. arXiv Preprint: 2309.04564 (cs.CL) (cit. on p. 24).

McKinney, W. (2010). “Data Structures for Statistical Computing in Python.” *Proceedings of the 9th Python in Science Conference*. Ed. by S. van der Walt and J. Millman, pp. 56–61. doi: 10.25080/Majora-92bf1922-00a (cit. on p. 34).

Muennighoff, N., A. Rush, B. Barak, T. Le Scao, N. Tazi, A. Piktus, S. Pyysalo, T. Wolf, and C. A. Raffel (2023). “Scaling data-constrained language models.” *Advances in Neural Information Processing Systems* 36, pp. 50358–50376 (cit. on p. 3).

Neudecker, C., K. Baierer, M. Gerber, C. Clausner, A. Antonacopoulos, and S. Pletschacher (2021). “A survey of OCR evaluation tools and metrics.” *Proceedings of the 6th International Workshop on Historical Document Imaging and Processing*, pp. 13–18 (cit. on p. 8).

Ney, H., U. Essen, and R. Kneser (1994). “On structuring probabilistic dependences in stochastic language modelling.” *Computer Speech & Language* 8.1, pp. 1–38. ISSN: 0885-2308. doi: <https://doi.org/10.1006/csla.1994.1001>. URL: <https://www.sciencedirect.com/science/article/pii/S0885230884710011> (cit. on p. 48).

Nguyen, T. T. H., A. Jatowt, M. Coustaty, and A. Doucet (2021). “Survey of post-OCR processing approaches.” *ACM Computing Surveys (CSUR)* 54.6, pp. 1–37 (cit. on p. 45).

OpenAI (2025). *GPT-OSS-120B & GPT-OSS-20B Model Card*. [online]. doi: [10.48550/arXiv.2508.10925](https://arxiv.org/abs/2508.10925). arXiv Preprint: [2508.10925](https://arxiv.org/abs/2508.10925) (cs.CL) (cit. on p. 10).

OpenAI (2026). *tiktoken*. [GitHub Code Repository]. URL: <https://github.com/openai/tiktoken> (cit. on p. 29).

Pedregosa, F., G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay (2011). “Scikit-learn: Machine Learning in Python.” *Journal of Machine Learning Research* 12, pp. 2825–2830 (cit. on p. 34).

Qwen Team (2025). *Qwen3 Technical Report*. [online]. doi: [10.48550/arXiv.2505.09388](https://arxiv.org/abs/2505.09388). arXiv Preprint: [2505.09388](https://arxiv.org/abs/2505.09388) (cs.CL) (cit. on pp. 10, 24, 54).

Raffel, C., N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu (2020). “Exploring the limits of transfer learning with a unified text-to-text transformer.” *Journal of Machine Learning Research* 21.140, pp. 1–67 (cit. on p. 45).

Rigaud, C., A. Doucet, M. Coustaty, and J.-P. Moreux (2019). “ICDAR 2019 competition on post-OCR text correction.” *2019 International Conference on Document Analysis and Recognition (ICDAR)*. IEEE, pp. 1588–1593 (cit. on p. 45).

Sadvilkar, N. and M. Neumann (Nov. 2020). “PySBD: Pragmatic Sentence Boundary Disambiguation.” *Proceedings of Second Workshop for NLP Open Source Software (NLP-OSS)*. Association for Computational Linguistics, pp. 110–114. doi: [10.18653/v1/2020.nlposs-1.15](https://arxiv.org/abs/2010.08001) (cit. on p. 52).

Sennrich, R., B. Haddow, and A. Birch (Aug. 2016). “Neural Machine Translation of Rare Words with Subword Units.” *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Ed. by K. Erk and N. A. Smith. Berlin, Germany: Association for Computational Linguistics, pp. 1715–1725. doi: [10.18653/v1/P16-1162](https://arxiv.org/abs/1608.05424). URL: <https://aclanthology.org/P16-1162/> (cit. on p. 54).

Senuma, H. (Jan. 2025). “mmh3: A Python extension for MurmurHash3.” *Journal of Open Source Software* 10.105. mmh3 code at <https://github.com/hajimes/mmh3>, p. 6124. ISSN: 2475-9066. doi: [10.21105/joss.06124](https://arxiv.org/abs/2501.06124) (cit. on p. 47).

Tange, O. (Feb. 2011). “GNU Parallel - The Command-Line Power Tool.” *login: The USENIX Magazine*, pp. 42–47 (cit. on p. 34).

The Pandas Development Team (Feb. 2020). *pandas-dev / pandas: Pandas*. doi: [10.5281/zenodo.3509134](https://doi.org/10.5281/zenodo.3509134) (cit. on p. 34).

The pyfranc development team and cyb3rk0tik (2023). *pyfranc*. <https://github.com/cyb3rk0tik/pyfranc>. [GitHub Code Repository] (cit. on p. 11).

The Unicode Consortium (Sept. 2022). *The Unicode Standard, Version 15.0 - Core Specification*. Tech. rep. Available at <https://www.unicode.org/versions/Unicode15.0.0/UnicodeStandard-15.0.pdf> (cit. on p. 8).

Tong, X. and D. A. Evans (1996). “A statistical approach to automatic OCR error correction in context.” *Fourth Workshop on Very Large Corpora* (cit. on p. 45).

Touvron, H., T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample (2023). *LLaMA: Open and Efficient Foundation Language Models*. [online]. doi: [10.48550/arXiv.2302.13971](https://doi.org/10.48550/arXiv.2302.13971). arXiv Preprint: [2302.13971](https://arxiv.org/abs/2302.13971) (cs.CL) (cit. on p. 45).

Tulkens, S. and T. van Dongen (2024). *Model2Vec: Fast State-of-the-Art Static Embeddings*. [GitHub Code Repository]. doi: [10.5281/zenodo.17270888](https://doi.org/10.5281/zenodo.17270888). URL: <https://github.com/MinishLab/model2vec> (cit. on p. 7).

Twine, N. (1984). “The adoption of punctuation in Japanese script.” *Visible Language* 18.3 (cit. on p. 17).

Vanderbauwhede, W. (2023). *Frugal Computing—On the need for low-carbon and sustainable computing and the path towards zero-carbon computing*. [online]. doi: [10.48550/arXiv.2303.06642](https://doi.org/10.48550/arXiv.2303.06642). arXiv Preprint: [2303.06642](https://arxiv.org/abs/2303.06642) (cs.CY) (cit. on p. 5).

Wang, J. and J. Wang (2015). “qRead: A fast and accurate article extraction method from web pages using partition features optimizations.” *2015 7th International Joint Conference on Knowledge Discovery, Knowledge Engineering and Knowledge Management (IC3K)*. Vol. 1. IEEE, pp. 364–371 (cit. on p. 45).

Weninger, T., W. H. Hsu, and J. Han (2010). “CETR: content extraction via tag ratios.” *Proceedings of the 19th international conference on World wide web*, pp. 971–980. doi: [10.1145/1772690.1772789](https://doi.org/10.1145/1772690.1772789) (cit. on p. 45).

Wenzek, G., M.-A. Lachaux, A. Conneau, V. Chaudhary, F. Guzmán, A. Joulin, and E. Grave (2020). “CCNet: Extracting high quality monolingual datasets from web crawl data.” *Proceedings of the Twelfth Language Resources and Evaluation Conference*, pp. 4003–4012 (cit. on pp. 24, 45).

Whistler, K. (July 2025). *Unicode Normalization Forms*. Unicode Standard Annex 15. Revision 57: <https://www.unicode.org/reports/tr15/tr15-57.html>. The Unicode Consortium (cit. on p. 8).

Wolf, T., L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, J. Davison, S. Shleifer, P. von Platen, C. Ma, Y. Jernite, J. Plu, C. Xu, T. L. Scao, S. Gugger, M. Drame, Q. Lhoest, and A. M. Rush (Oct. 2020). “Transformers: State-of-the-Art Natural Language Processing.” *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, pp. 38–45. doi: [10.18653/v1/2020.emnlp-demos.6](https://doi.org/10.18653/v1/2020.emnlp-demos.6) (cit. on p. 54).

Xie, Q., Z. Dai, E. Hovy, T. Luong, and Q. Le (2020). “Unsupervised data augmentation for consistency training.” *Advances in Neural Information Processing Systems* 33, pp. 6256–6268 (cit. on p. 10).

Zhu, E., V. Markovtsev, A. Astafiev, A. Khan, C. Ha, W. Łukasiewicz, A. Foster, Sinusoidal36, S. Thakur, S. Ortolani, Titusz, V. Letal, Z. Bentley, fpug, hgulich, long2ice, oisincar, R. Assa, S. Ibraimoski, R. Kumar, Q. TianHuan, M. J. Rosenthal, K. Joshi, K. Mann, JonR, J. Halliwell, and A. Oriekhov (June 2024). *ekzhu/datasketch: v1.6.5*. Version v1.6.5. [Code Repository]. doi: [10.5281/zenodo.11462182](https://doi.org/10.5281/zenodo.11462182) (cit. on pp. 14, 50).

Appendices

A Dataset Fields

Field name	Type	Description	Section
barcode_src	string	The volume’s barcode. Serves as a primary key/identifier and join key to IB-HL.	–
primary_language_gen	string	ISO 639-3 code for the main language of this volume, taken from IB-HL for convenience.	–
language_distribution_gen	list[(string, float)]	Pairs of (LANG, PROPORTION) where LANG is an ISO 639-3 code and PROPORTION is the 0–1 fraction of the paragraphs in the volume with that language. Only languages with ≥ 5 paragraphs contribute.	§4.12
token_count_gen	int	Total tokens for that volume’s middlematter, as measured with o200k_BASE.	§4.12
char_count_gen	int	Total characters in middlematter.	§4.12
word_count_gen	int	Total words in middlematter (language-aware tokenization).	§4.12
sentence_count_gen	int	Total sentences in middlematter (language-aware segmentation).	§4.8
paragraph_count_gen	int	Total subtopic paragraphs in middlematter.	§4.9
section_count_gen	int	Total subtopic sections in middlematter.	§4.9
bigram_count_gen	int	Total word bigrams in middlematter.	§4.12
bigram_count_unique_gen	int	Distinct word bigrams in middlematter.	§4.12
trigram_count_gen	int	Total word trigrams in middlematter.	§4.12
trigram_count_unique_gen	int	Distinct word trigrams in middlematter.	§4.12
tokenizability_ratio_gen	float (0.0–100.0)	Measure of how close this text is to 1.25 o200k_BASE tokens per word.	§4.12
bpb_{min,max,median,avg}_gen	float	Per-volume bits-per-byte summary over scored paragraphs.	§4.11
bpb_p{10,30,70,90}_gen	float	Per-volume bits-per-byte percentiles.	§4.11
frontmatter_gen	string	Annotated frontmatter: contains endmatter divs.	§4.4 and §4.12
middlematter_gen	string	Annotated body: contains sections, paragraphs, and bpb/language/duplicate tags.	§4.12
backmatter_gen	string	Annotated backmatter: contains endmatter divs.	§4.4 and §4.12
processed_middlematter_gen	string	Middlematter filtered to non-duplicate paragraphs in the [p10, p90] bpb band.	§4.12

Table 23: Fields in the dataset.

The suffix _src denotes fields taken directly from source inputs. The suffix _gen denotes values computed by this pipeline. All text statistics are computed over middlematter.

B Additional related work

Our work incorporates many different active areas of research and engineering. We indicate relevant work, especially work that informed our design, even if we do not use it directly.

Cleaning Datasets. Computationally inexpensive boilerplate detection is well studied (Weninger, W. H. Hsu, and Han 2010; J. Wang and J. Wang 2015; Kohlschütter, Fankhauser, and Nejd 2010). OCR post-analysis and detection is its own field of research, with some methods based on statistical analysis (Kukich 1992; Tong and Evans 1996; Jatowt et al. 2019). See T. T. H. Nguyen et al. 2021 for a survey describing different approaches. More recently, modern ML-based techniques (especially models based on transformers) have been used. ICDAR held competitions in 2017 and 2019 on OCR correction (Chiron et al. 2017; Rigaud et al. 2019) in which neural networks and BERT-based models (Devlin et al. 2019) outperformed most other methods.

Data Curation. Assembling public datasets for model training is a multistage process that can include data sourcing, deduplication, cleaning, and filtering. The C4 dataset (Raffel et al. 2020) is a large-scale dataset from internet-scraped data, processed with heuristic cleaning, English language restriction, and chunk-based deduplication.

One difficulty in this area is that details about the training process and data for state-of-the-art language models are often opaque. Even open-weight models such as LLaMA, Mistral, Gemma, GLM, and Kimi reveal little about underlying training data (Touvron et al. 2023; Jiang et al. 2023; Gemma Team et al. 2025; GLM-5-Team et al. 2026; Kimi Team et al. 2025). To improve transparency and to facilitate comparison, DCLM (J. Li et al. 2024) establishes benchmarks for data curation.

Landscape Surveys. The progression from large data curation (Gao et al. 2020; Raffel et al. 2020), to filtering and extraction (Wenzek et al. 2020), to post-creation analysis (Dodge et al. 2021) helped shape our approach to data. Datacomp-LM (J. Li et al. 2024) also provides a thorough literature review and survey of closely related areas.

C Details on Processing

This appendix contains additional technical details and design rationale.

C.1 Preprocessing Details

Language-based Models. This setup relies on language reporting from IB-HL, as we use that language detection for stratification. Each of the models here is a *base* model and is later updated for each book processed. A small amount of noise in the base model has minimal effect on the resulting behavior in our pipeline. An additional source of noise comes from training base models on pre-cleaned text. For most languages and most volumes, however, the majority of the text has sufficiently little noise to allow effective use of small base-language models.

Segmenter-based Partitioning. The IB-HL dataset can be naturally streamed in shards of 200 books. Rather than use those shards, we immediately split shards into NUPUNKT-compatible languages and non-NUPUNKT-compatible languages. We do this because NUPUNKT-compatible shards can be processed entirely on CPUs, while the other shards require GPUs to complete sentence segmentation in a reasonable amount of time. After this sharding, all further processing (except deduplication) is completely independent.

C.2 Unicode Normalization

Soft and Hard Normalization. See Listing 24 for the exact steps for both “soft” and “hard” normalization. We apply NFC and NFKC normalization via `unicodedata.normalize` in the Python standard library. The remaining steps are strict string replacement and regular expression operations.

Soft Unicode Normalization 1. NFC Unicode Normalization 2. Remove zero-width characters: U+200B, U+FEFF 3. Condense Unicode space characters: Map U+00A0, U+2000-U+200A, U+202F, U+205F, and U+3000 to ASCII space 4. Collapse consecutive spaces 5. Strip per-line leading/trailing spaces PRESERVES: line breaks, curly quotes, hyphens, dashes, ligatures, accents.	Hard Unicode Normalization 1. NFKC Unicode Normalization 2. Remove zero-width characters: U+200B, U+FEFF 3. Map newlines '\r\n' and '\r' to '\n' 4. Map hyphen/dash-like to ASCII dash U+2010-U+2015, U+2212, U+FE58, U+FE63, U+FF0D to ASCII dash. 5. Remove U+00AD soft-hyphen. 6. Map curly quotes to ASCII quote U+2018-U+201B and backtick to ' U+201C-U+201F to '" 7. Map Unicode space characters to ASCII space U+00A0, U+2000-U+200A, U+202F, U+205F, and U+3000 8. Flatten tabs and newlines to ASCII space 9. Collapse consecutive spaces 10. Strip per-line leading/trailing spaces OUTPUT: a single line with ASCII punctuation and NFKC-folded characters.
--	---

Listing 24: Explicit Unicode Normalization

Case	Input	Soft	Hard
Precomposed vs. combining accent	e + U+0301	é (NFC)	é
Ligature	U+FB01 (fi)	fi (kept)	fi
Full-width letter	U+FF21 (A)	A (kept)	A
Superscript digit	U+00B2 (²)	² (kept)	2
Curly quotation marks	U+201C ... U+201D	" ... "	" ... "
En / em dash, minus sign	U+2013 / U+2014 / U+2212	— / — / −	—
Non-breaking space	U+00A0	ASCII space	ASCII space
Zero-width space	U+200B	(removed)	(removed)

Table 25: Soft vs. hard Unicode normalization on boundary cases.

Zero-Width Spaces. One aspect of zero-width spaces merits additional discussion. The zero-width spaces U+200C (zero-width no-join) and U+200D (zero-width yes-join) are kept, despite not being visible. These are mainly used in cursive Arabic, Persian, and Indic scripts to either prevent characters joining in cursive or to force joining in cursive. These zero-width spaces can help distinguish word or morpheme boundaries and carry semantic meaning. (U+200D is also used in some emoji sequences, but this is not relevant for this corpus). As OCR continues to improve for cursive Arabic, Persian, and Indic scripts, these zero-width formatting spaces will likely become more prevalent.

Boundary Examples. Table 25 contrasts the “soft” and “hard” normalization on representative inputs.

C.3 Duplicate Page Removal

For duplicate page removal, we first hard-normalize each page in a volume (cf. §4.2). Thus each page is temporarily reduced to a single long string of text for duplicate comparison.

Minimum Page Length. To ensure that pages have sufficient text to be worth checking for duplicates, we only consider pages with at least 50 non-whitespace Unicode code points (after hard-normalization). Technically this carries a small bias against detecting duplicates in languages that have more semantic meaning per character. For example, 50 Chinese characters likely carry more semantic content than 50 ASCII characters. This is a mild lower bound that prevents excessive removal of short content.

N-Gram Length. The simhashes are computed over character 9-grams. Longer n-grams make the hash more sensitive to exact content and less prone to random short common phrase collision. Shorter n-grams allow more n-grams and random mixing per page. We chose 9 to balance these competing requirements.

Compiled C++ Simhash Extension. We use the MurmurHash3 non-cryptographic hash function for rapid hashing; it is faster than cryptographic hash functions such as SHA256, and cryptographic security is not required. MurmurHash3 was specifically designed for high performance after compilation (Appleby 2016). The standard implementation for Python, `mmh3` (Senuma 2025) is written in C and computes hashes quickly, but passing Python objects to `mmh3` and manipulating the hashes in Python adds overhead.

We include a lightly customized version of MurmurHash3 in an optimized C++ simhash implementation. This implementation is designed for 64-bit platforms (little-endian only), uses GCC/Clang optimized `POPCNT` for rapid Hamming distance calculation between simhashes, and does not need to pass objects back to Python for comparison. Benchmarks show that this results in an approximately 100-fold speedup over a direct Python implementation of simhash using `mmh3`. We verify that our MurmurHash3 implementation agrees with `mmh3` in a test suite.

Probability Analysis. Page simhashes that differ by 6 or fewer bits are recognized as duplicates. This is a strict threshold and should rarely occur by chance. In detail: for any fixed 128-bit integer, the number of other 128-bit integers with Hamming distance at most 6 is

$$B(128, 6) = \sum_{k=0}^6 \binom{128}{k} \approx 5.70 \times 10^9.$$

Thus the probability that two random hashes will have Hamming distance at most 6 is approximately

$$B(128, 6)/2^{128} \approx 1.67 \times 10^{-29}.$$

For a collection of N hashes, the expected number of accidental 6-bit near-collisions is approximately $\binom{N}{2} \cdot 1.67 \cdot 10^{-29}$. Hence even for the largest books, the expected number of accidental collisions is negligible.

C.4 Endmatter Separation

Compute Parameters. Pages are batched, but at most from a single book at a time. We found empirically that batches of size ≈ 1024 are optimal (capable of 7200 pages/s per CPU core). Book page lengths resulted in our typical batch size being closer to 350 (6800 pages/s per CPU core). In practice, total memory bandwidth is a limiting factor and using 20 cores is not 20 times faster than using 1 core.

Bottleneck. Benchmarks show that the embedding table of dimension 512 limits overall speed. The embedding table used by `MODEL2VEC` has a 250k vocabulary (inherited from BAAI/BGE-M3), each mapping to 512 dimensions with a 4-byte `float32`. Hence the embedding table is approximately 512MB and does not fit into cache on most CPUs. Most table access requires memory lookup.

Multilingual Generalization. We apply the same models to every book in every language. We believe that the multilingual training supporting BAAI/BGE-M3 helps provide good signal across the languages in IB-HL. For languages not covered in its training data, one can sometimes expect a small degree of zero-shot generalization for languages that share scripts or lexical features with those seen during training (Chen et al. 2024).

Generalized Training Data. The pipeline repository includes code that trains analogous classifiers based on given data. The repository allows configuring static models for `MODEL2VEC`, including the option to bypass static models entirely. The repository is designed to facilitate training classifiers on alternative data.

C.5 Dehyphenation

Two Models are Necessary. Both the base model and the per-book model are necessary. Per-book models lack statistics for long words that appear only once in a book. Long names typically appear in only a few books, and hence are not contained in base language statistical distributions. Combining per-book and per-language base statistics leads to more reliable dehyphenation decisions.

Probabilistic Interpretation. Our model is a “naive joint probability estimator with add- k smoothing and fixed *Stupid Backoff* (Brants et al. 2007) penalty.” We call it *naive* because all probabilities are assumed independent. The model is not a proper probability model because the backoff does not take into account the probability mass from full n-grams (distinguishing it from more advanced Katz-style backoff (S. Katz 1987)). Though independence is objectively false and the backoff model is simple, these simplifications have minimal effect on the relative ranking among the three possibilities in practice. It would not be appropriate, however, to rely on our probability estimator to generate text.

Alternatives Considered. Prototypes used the KenLM (Heafield 2011) implementation of a full Kneser-Ney smoothed n-gram language model (Ney, Essen, and Kneser 1994). This is a more precise and robust language model that offers proper conditional probabilities. However, KenLM was substantially more computationally expensive (adding ≈ 10 s per book). Experimenting with less computationally expensive alternatives showed that our simple joint probability estimator in nearly all cases generates the same relative ranking at substantially lower

```

SYSTEM_PROMPT = """
You create realistic book content pages that reflect both historical and linguistic
accuracy. Your task is to generate sample pages in a specified language, adhering to the
conventions of the given era.
"""

GOOD_PAGE_TYPES = [ "Fiction Novel", "Non-fiction", "Textbook Exposition",
    "Argumentative Essay", "Documentation from a how-to manual",
    "Dialogue from a play or interview", "Travel Writing", "Academic Writing",
    "Letters of Correspondence", "Poetry" ]
BAD_PAGE_TYPES = [ "Table of Contents", "Copyright", "Dedication Page", "Index Page",
    "Bibliography", "Section Dividers", "Frontmatter", "Backmatter" ]
ERAS = [ "pre1800s", "1800-1825", "1826-1850", "1851-1875",
    "1876-1900", "1901-1925", "modern", ]
GOOD_PROMPT = """
You will generate pages from TWO fictional books in the language '{language}' (ISO 639-3
code) set in the {era} era, but with contrasting content.

For EACH of the two books, generate ONE COMPLETE EXAMPLE PAGE found in a {page_type}
type of book.

Output requirements:

* Total pages: TWO (one for each book).
* After each page, insert a line with exactly: #####
* The content of each page must be in the target language only.
* Make the tone and layout appropriate to the genre of each book.
* Do not include any translations or meta-commentary.
* If you cannot write in the language for '{language}', output exactly UNSUPPORTED_LANGUAGE
"""

BAD_PROMPT = """
You will generate frontmatter and backmatter pages for TWO different fictional books in
the language '{language}' (ISO 639-3 code), both set in the {era} era, but with
contrasting genres.

First book: a formal, scholarly, or religious work.
Second book: an informal, narrative, or literary work (such as a novel, memoir, or story
collection).

For EACH of the two books, generate ONE COMPLETE EXAMPLE PAGE found in the {page_type}
of the book.

Output requirements:

* Total pages: TWO (one for each book).
* After each page, insert a line with exactly: #####
* The content of each page must be in the target language only.
* Make the tone and layout appropriate to the genre of each book.
* Do not include any translations or meta-commentary.
* If you cannot write in the language for '{language}', output exactly UNSUPPORTED_LANGUAGE
"""

```

Listing 26: Synthetic Data Prompt Structure

cost.

C.6 Running Headers and Footers

We tuned the Jaccard similarity threshold of 0.85 to allow approximately one typo or one digit to change in page numbers for a standard, relatively short header. Experiments show that this tends to be conservative in practice.

We did not optimize this portion of the pipeline. We use the default hash algorithm from the `DATASKETCH` library (Zhu et al. 2024), with default seeding and the SHA-1 hash function. It would be possible to use a fast hash function like MurmurHash3 (as in Appendix C.3), but the bottleneck is in LSH construction. For each n -gram, `DATASKETCH` performs a hash *and* a NumPy permutation-and-min operation. The permutation-and-min dominates timing.

C.7 Page Number Removal

Removing page numbers across different writing systems faces many edge cases. We rely on the Unicode category of the underlying symbol: if the Unicode category starts with “N”, we call it numeric. Otherwise it is non-numeric. This accepts bare numbers (123), numbers with a stray mark (p 42), Arabic-Indic numbers, Devanagari numbers, full-width Unicode numbers, Unicode encoded Roman numerals (such as U+2163 for IV instead of ASCII IV), and so on. It does not accept certain less common page number formats, such as dash-flanked numbers (– 12 –).

Most current OCR engines do not use Unicode encodings for Roman numerals and instead use ASCII letters. Another known gap comes from Han ideographic numerals. For example, ‘十 百’ are each classified in Unicode as (Letter, other) characters and *not* as numbers. Treating these ideographs as numbers (which is what Python’s `str.isnumeric()` does) would lead to recognizing legitimate lines such as ‘十年’ (“ten years”) as likely page numbers.

Likely OCR Risk. Several different scripts use nearly identical shapes with different meanings, e.g. the English digit “0” vs the Arabic letter “٠” vs the Devanagari digit “०” vs the Greek letter o vs a small circle $\circ$. Each OCR engine produces its best estimate from context and runtime configuration.

C.8 Segmentation Details

The list of NUPUNKT-compatible languages is in Table 27.

Determining NUPUNKT vs SAT languages. To choose between NUPUNKT and SAT models for sentence segmentation, we weighed accuracy and speed.

NUPUNKT examines punctuation and performs well when punctuation use resembles common European languages. SAT uses deep learning. If speed (or total compute) were not an issue, then we would use 12-layer SAT for all segmentation; small samples suggest that SAT has better segmentation in almost all cases. But SAT requires vastly larger amounts of compute.

We randomly sampled 10 books in each language from the collection (or as many as possible if 10 were not available) and used both NUPUNKT and SAT to segment into sentences. We assumed that SAT-3L-SM was “correct” and assessed how accurately NUPUNKT predicted the

als	arl	arn	ast	bel	bem	bin	bos	bre	cab
cak	cbt	ces	chk	chv	cic	cjk	ckb	cnh	cnr
cof	ctd	cym	dan	deu	dga	ekk	ell	eng	epo
eus	ewe	fao	fat	fij	fin	fkv	fra	fry	gla
gle	glg	glv	gyr	hat	haw	heb	hil	hlt	hns
hrv	hsb	hun	ibo	ido	ijs	ilo	isl	ita	kal
kat	kaz	kir	kjh	kmb	kng	koi	ktu	lat	lin
lit	lld	loz	lua	lug	lun	mad	men	mic	min
mlt	mri	nba	nbl	ndo	niu	njo	nld	nno	nya
nym	nyn	oki	oss	piu	plt	pol	por	pov	ppl
que	qug	rar	roh	ron	rus	sah	sco	slk	slv
sme	snk	spa	srp	suk	sun	sus	swb	swe	swh
tam	tat	tgl	tsn	tso	tuk	tur	ukr	ura	ven
vie	war	xho	yao	ykg	yua	zro	zul		

Table 27: NUPUNKT-compatible languages, given as ISO 639-3 strings

indices for starts of sentences. Treating the output of SaT as “correct” is only a heuristic. In the unlikely event that a language is well-segmented by NUPUNKT but poorly segmented by SaT, this heuristic would cause the worse segmenter to be chosen.

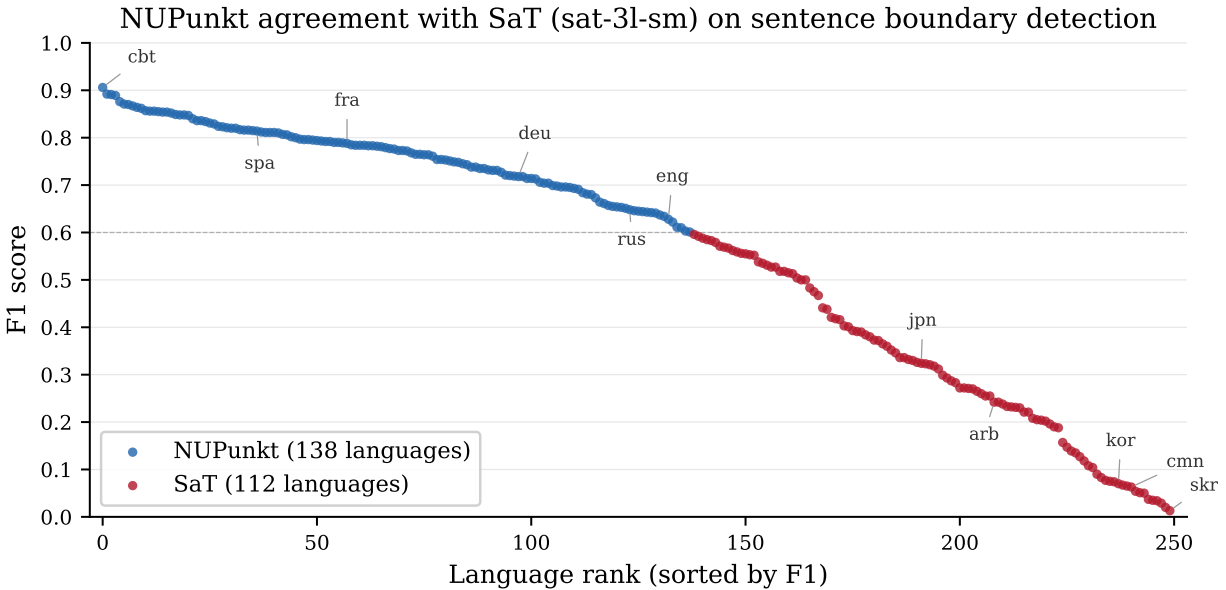


Figure 28: F1 scores in decreasing order with certain notable languages indicated.

A plot of the resulting F1 scores (on identifying SaT sentence start indices) is in Figure 28. Instead of a sharp drop, NUPUNKT performance gradually decreases. Benchmarks suggest that both NUPUNKT and SaT perform well on English. We chose the cutoff between NUPUNKT and SaT to be at the 0.6 F1 score. English is close to the cutoff, suggesting that NUPUNKT should segment all “NUPUNKT-compatible” languages approximately as well as it segments English. This cutoff also guaranteed that the vast majority of books in IB-HL would be segmented via NUPUNKT, allowing a predominantly CPU-based pipeline.

NUPUNKT per-book adaptation. To estimate the value of NUPUNKT per-book adaptation, we ran experiments. We took 195 books (15 books in each of English, French, Portuguese, Spanish, Polish, Russian, Italian, Latin, Dutch, German, Danish, Greek, and Swedish) and examined concordance between detected sentence boundaries between each of (1) base language model NUPUNKT, (2) per-book adapted NUPUNKT, and (3) SAT. (The setup is similar to the setup leading to Figure 28).

In general, we found that base-language NUPUNKT models and per-book adapted NUPUNKT models agree approximately 93% of the time, while per-book adapted NUPUNKT models agree slightly more with SAT. Manual checking suggests that the 7% difference with per-book adapted NUPUNKT models corresponds to more accurate segmentation.

We conclude that per-book adaptation yields a notable improvement, but may be omitted when compute is constrained.

More on NUPUNKT vs SAT speed comparisons. Base-language NUPUNKT models perform about twice as fast as SAT in our benchmarks. Further, on machines with multiple CPU cores, multiple NUPUNKT models can operate in parallel. We observed that one can run up to 8 simultaneous NUPUNKT inference processes on a DGX Spark before confronting memory bandwidth or RAM limitations. However, including per-book adaptation changes this tradeoff: 8 simultaneous NUPUNKT adaptation-and-inference processes might segment books at comparable speeds to a single reasonably fast GPU. Choosing which path and model to use thus depends heavily on whether one has access to GPUs.

We estimate that a single NVIDIA A100 could perform SAT segmentation on all of IB-HL in about 1 month. For comparison, we estimate that a single DGX Spark could perform the current SAT+NUPUNKT split segmentation in approximately 2 weeks; this decreases to 3 days when omitting per-book adaptation.

Alternative Segmenters Considered. We ran the segmentation-vs-SAT experiment shown in Figure 28 with 4 other libraries in all languages and 1 additional library on selected languages. We briefly comment on these.

1. MWTokenizer²⁵, a multilingual Python-based tool used widely on Wikipedia. It ran faster than NUPUNKT and had comparable behavior on Western European languages, but was inconsistent with other languages.
2. SENTENCEX²⁶, another segmenter used by Wikimedia. It achieved excellent accuracy but was too slow for this application.
3. PYSBD²⁷ “Pragmatic Sentence Boundary Disambiguation” (Sadvilkar and Neumann 2020). It had strong performance on 15 languages, but was over 10× slower than NUPUNKT. Experiments suggest that its runtime grows super-linearly in text length, complicating efforts to segment book-length volumes.
4. BLINGFIRE2²⁸, developed by the Microsoft BLING (Beyond Language understanding) team. Performance and speed are comparable to NUPUNKT.

²⁵<https://gitlab.wikimedia.org/repos/research/wiki-nlp-tools/>

²⁶<https://github.com/wikimedia/sentencex>

²⁷<https://github.com/nipunsadvilkar/pySBD>

²⁸<https://pypi.org/project/blingfire2/>

5. `NLTK LIBRARIES`²⁹ for the Python natural language toolkit. Most languages have at least one high-performing language-specific NLTK library, but there are many and we did not allocate time to test hundreds of NLTK libraries. The default English segmenter was less reliable than `NUPUNKT`.

One distinct advantage of `NUPUNKT` is that per-book unsupervised learning leads to improvements in segmentation quality.

A different alternative would be to use `SaT` on more languages. Annotated training data can be used to fine-tune `SaT` models to particular languages. In an ideal scenario, we would use high-quality training data in low-resource languages to generate more reliable outputs.

On the Lack of `SaT` Batching. The `SaT` library allows batching, but we do not batch the books. This is because the typical book text is long, and internally `SaT` splits long inputs into overlapping 512 token windows and batches those tokens through the model. A single medium-length book can saturate a GPU on its own. Small batch testing suggests that batching books into groups of 200 (and allowing `SaT` to generate appropriate batches internally) improves throughput between 15 and 25 percent. This is a notable increase, but does not justify the added complexity for only 20,629 books. Instead, the pipeline is organized for simple error tracking and resuming after faults.

C.9 Alternate Chunking Algorithms Considered

Many high-quality chunking algorithms require a GPU in practice. For example, one can use a BERT or RoBERTa model fine-tuned on training data. We did not find any GPU-based chunking algorithm that fit within our compute constraints using available multilingual training data.

A priori, we expected a small variant of the sentence segmentation algorithms to be serviceable. Some sentence segmentation algorithms assign probabilities for tokens to be the end of a sentence. We hypothesized that choosing a high threshold for this probability could break chunks of sentences into paragraphs, but experiments showed that this approach did not work well in practice.

We found `TextTiling` (Hearst 1997) and `C99` (Choi 2000) to be computationally viable. When combined with modern multilingual sentence embeddings, they are both performant and customizable. The pipeline code repository contains complete modern implementations of both a `TextTiling`-based algorithm and a `C99`-based algorithm. We ultimately chose `TextTiling` because it led to better duplicate detection later in the pipeline.

C.10 Duplicate Identification Details

Band Bucket Sizes. Each 128-bit simhash is split into 6 bands of (22, 21, 21, 21, 21, 22) bits. We consider two hashes with Hamming distance at most 5 to be duplicates. Any such pair must have at least one identical band, and hence to identify candidate duplicate pairs we search for exact matches across each of the 6 bands.

Each band has at least 21 bits and there are $\approx 1.4 \times 10^9$ paragraphs. Each band holds on

²⁹<https://www.nltk.org>

average

$$\frac{\text{\# paragraphs}}{\text{\# possible values}} \approx \frac{1.4 \cdot 10^9}{2^{21}} \approx 667$$

different paragraphs. Thus after restricting to buckets formed from exact band equality, buckets are small enough on average to allow exhaustive comparison (each requiring two xors, two popcounts, and one addition over 64-bit limbs). We implemented an upper limit: if a bucket has more than 30k items, it logs a warning and does not compare within the bucket. This limit is never reached for this collection.

Resource Footprint. The global simhash array (`hashes.bin`, $\approx 23\text{GB}$) is memory-mapped and available to individual worker processes. The union-find data structure is managed by a parent process and individual workers require almost no private state in addition to access to the page-cached simhash array. Each worker computes Hamming distances in parallel and streams confirmed pairs to the parent process. Confirmed near-duplicate pairs are merged with a union-find data structure keyed on the transient flat paragraph index. This structure stores up to one index per paragraph, as well as up to one byte to track rank for each paragraph ($1.4\text{B entries} \times (8 \text{ byte index} + 1 \text{ byte rank})$, roughly 13GB in total). The duplicate identification pipeline fits in 48GB of RAM.

C.11 Bits-Per-Byte Details

The bits-per-byte computation is a thin wrapper around direct calls on the tokenizer and model from QWEN/QWEN3-0.6B-BASE using the TRANSFORMERS library and PyTorch (Qwen Team 2025; Wolf et al. 2020; Ansel et al. 2024). The implementation closely follows standard PyTorch and TRANSFORMERS usage, except that QWEN3 models require right padding rather than left padding during batching.

Determining the Language Model. We initially sought an inexpensive but reliable proxy for perplexity (before switching to per-byte normalization for BPB instead of per-token normalization). We investigated whether easily computable features could accurately predict perplexity.

To make this concrete, we studied whether these features allow accurate prediction of QWEN3 perplexity across models of different sizes ranging from 0.6B parameters to 14B parameters. We trained simple Ridge linear regression models on an 80-20 train-test split from 3067 random pages of text from IB-HL to explore feasibility.

1. N-gram complexity from a complete-but-efficient n-gram language model like KenLM (Heafield 2011) in a few variants:
 - character n-grams,
 - word n-grams based on SentencePiece (Kudo and Richardson 2018) subword units (Sennrich, Haddow, and Birch 2016), or
 - QWEN3 tokenizer n-grams (where n-grams consist of sets of n tokens coming from a QWEN3 tokenizer).
2. Embeddings, predictions, and confidences from the Endmatter classifier (cf. §4.4),
3. Fundamental natural language processing descriptors, including
 - character count,
 - word count,

- ratio of characters to newlines,
- ratio of characters to whitespace,
- ratio of characters that are digits,
- number of unique chars,
- number of bigrams,
- bigram entropy, and
- rare bigram ratios.

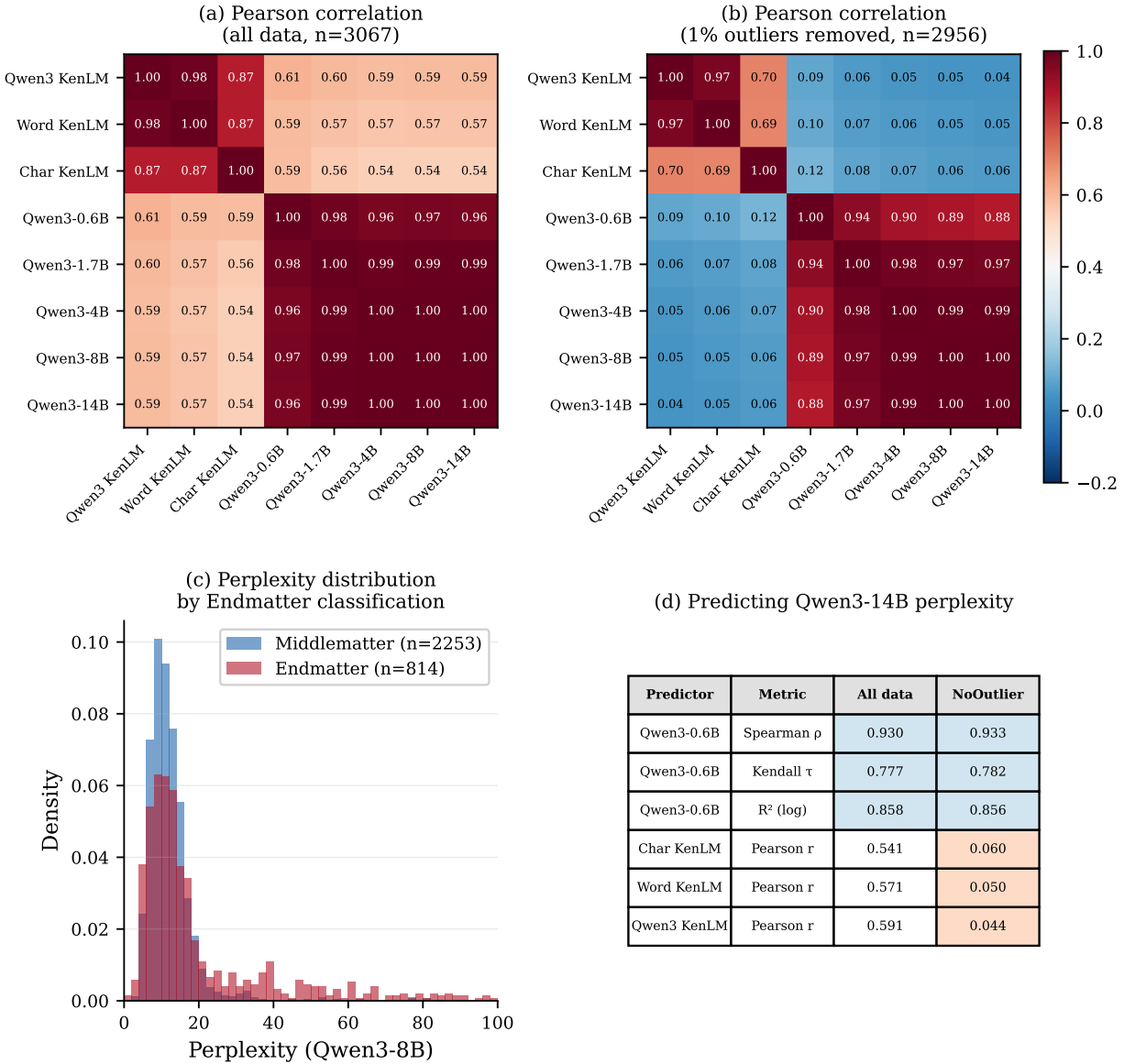


Figure 29: Statistical correlations of QWEN3 perplexity with other measures

See Figure 29 for correlations and prediction behavior. We observed that multiple features carried predictive power for extremely high perplexity (especially the rare bigram ratio and all KenLM language models). After removing the 1% outliers,³⁰ we observed almost no cor-

³⁰We removed the top and bottom 1% for each of the three ngram metrics for panel (b). Many pages were simul-

relation between actual perplexity and estimated perplexity (panel (b)). The bimodal aspect of endmatter classification leads to poor predictive behavior (though *extreme* perplexity does weakly correlate with being endmatter).

We did not anticipate the difference in behavior between panel (a) and panel (b) in Figure 29. The predictions from n-gram language models correlate strongly with each other (even if they use different atomic units of text), but do not correlate with actual perplexity. One should expect QWEN3 models with similar parameter counts to have higher correlation, but it was not obvious that the smallest model, 0.6B, was highly predictive of the 14B model. Further, panel (d) shows that the relative ordering between perplexities is largely preserved (Kendall $\tau = 0.782$). From this, we determined that we would use the smallest, most computationally efficient model.

Error Rates in Computing BPB. After running the full pipeline, we noted that a different implementation of computing cross-entropy would allow effective computation of BPB values even for long paragraphs. Specifically, one could implement *chunked* or *fused* cross-entropy (P.-L. Hsu et al. 2024). Alternatively, one could use multiple GPUs together instead of many independent GPUs. We did not revisit the BPB values of the 4 too-short or the 12,391 too-long paragraphs.

D Examples with Known Limitations

Though we have tried to be conservative in cleaning and applying changes, we know of examples where the current pipeline has unintended side effects. We give two examples here.

Dehyphenation and Mathematics. The current pipeline does not treat math differently from prose (nor does the currently available OCR). In technical texts, minus signs and fraction bars might be treated as end-of-line hyphens and potentially removed during dehyphenation (§4.5). For example, Figure 30 is taken from *A first course in the differential and integral calculus* (barcode 32044000046607), page 448 (with page number 426 in the book).

Figure 30 shows three lines of exercises from a book, the source OCR output (including original newlines), and the output after dehyphenation. Three changes in hyphenation occur in this example:

1. The lone hyphen on the second line of the source OCR is removed.
2. The hyphen at the end of the exercise 34 line in the OCR text is removed, causing $x - 1 - \sqrt[n]{3}$ to be replaced by $x - 1 \sqrt[n]{3}$.
3. The lone hyphen on the line after exercise 35 in the source OCR is kept, but the newline is removed and now the $4x-7$ on the following line becomes $-4x-7$.

All three affect the meaning of the source OCR. But the source OCR does not capture the full meaning of the original math exercises.³¹

taneous outliers, leading to the removal of only 111 pages.

³¹ One might ask where the three hyphens came from. The first likely originated from the fraction bar in either $\frac{1}{3}$ from exercise 33 or $\frac{x^2}{3}$ from exercise 34; the second has to do with the OCR engine determining that $\frac{4x-7}{9}$ belongs

$$33. \frac{1}{3}x^3 - 2(a-b)x^2 + 3bx - a + b.$$

$$34. \frac{x^3}{3} - 2(3x-5) + x - 1 - \frac{4x-7}{9}.$$

$$35. (3-2x)^7$$

$$36. (p+qx)^n.$$

33. 2-2 (a - b) x²+3 bx - a+b.
-
34. 1 - 2(3x-5) + x - 1 -
3
35. (3-2x)
-
4x-7

Source OCR Text

33. 2-2 (a - b) x²+3 bx - a+b.
34. 1 - 2(3x-5) + x - 1 3
35. (3-2x)
-4x-7

Dehyphenated Text

Figure 30: Example of Meaning-Altering Dehyphenation Taken from 32044000046607.

These cases are rare relative to the billions of correct merges. But these examples suggest that technical and mathematical volumes are disproportionately affected.

Removing Citation Footers. Header/footer removal in this pipeline targets repetitive lines near the top or bottom of many nearby pages. Some books use abbreviated citation formats in footers or marginalia. When many nearly-identical citations occur in close proximity, they may all be removed. Figure 31 shows three footers of pages in *Geschichte der K. und K. Technischen Militär-Akademie* (barcode 32044004475331).

These footnotes include repeated references to K. und k. Kriegs-Archiv., and Ebendort.. Similar citation patterns occur throughout the book. As these lines recur at the foot of the page, the remover treats them as running footers. The number of citations to Kriegs-Archiv decreases from 137 occurrences in the source OCR to 64 in the output.

Currently all marginalia are OCRed and grouped with the main text, causing remaining bibliographic references to be placed in the middle of paragraphs. Current treatment of marginalia at both the OCR level and the cleaning level does not allow accurate tracking of bibliographic provenance. We regard restoring these paratextual elements as a worthwhile goal for future work.

E Using the Parser Library

We wrote a lightweight, pure-Python, dependency-free parser library³² and tool that can be used to filter and iterate through the final annotated dataset. The library uses `html.parser.HTMLParser` from the Python standard library as the core HTML parser.

The library exposes its functionality through the `BookDataset` class, which takes any iterable

in a column with exercise 36, so ordering is confusing; and the third is probably the misplaced fraction bar from $\frac{4x-7}{9}$.

³²<https://github.com/institutional/institutional-books-enriched-text-parser>

in einer Angelegenheit der Ingenieur-Akademie traf. ¹⁾ Acten der k. und k. Technischen Militär-Akademie. ²⁾ K. und k. Kriegs-Archiv. ³⁾ Ebendort. ⁴⁾ Ebendort.
»Portionen überkommen sollen.«³⁾ ¹⁾ K. und k. Kriegs-Archiv. ²⁾ Acten der k. und k. Technischen Militär-Akademie. ³⁾ Ebendort.
Akademie-Territorium nahezu vollendet. Am 30. März 1787, also noch ¹⁾ K. und k. Kriegs-Archiv. ²⁾ Acten der k. und k. Technischen Militär-Akademie. ³⁾ K. und k. Kriegs-Archiv. ⁴⁾ Acten der k. und k. Technischen Militär-Akademie.

Figure 31: Example of Removing Citations in Footers Taken from 32044004475331.

Object field	Type	Description
paragraph.text	string	Content of the paragraph
paragraph.bpb	float	QWEN/QWEN3-0.6B-BASE computed bits-per-byte
paragraph.language	string	ISO 639-3 code of language detected for the paragraph
paragraph.is_duplicate	boolean	Whether this paragraph is a duplicate of another paragraph in the whole collection.
section.paragraphs	list[paragraph]	Paragraph objects contained in section
section.bpb	float	Average bits-per-byte of contained paragraphs
book.bpb.{p10,p30,median,p70,p90,avg}	float	BPB percentile and average values

Table 32: IBET Parser Object Description

of Python dictionaries with a schema matching the dataset fields described in Appendix A. This can take many forms, such as a list of manually curated dictionaries, partial outputs from the pipeline, or the streamed HuggingFace dataset (Lhoest et al. 2021) produced by this pipeline. For example:

```

from datasets import load_dataset
from ibet_parser import BookDataset

ds = load_dataset(
    "institutional/institutional-books-hl-enriched-text",
    split="train", streaming=True
)
books = BookDataset(ds)

```

Once BookDataset is instantiated, one can filter and iterate through books, sections, and paragraphs:

```

from itertools import islice

for book in islice(books, 10):      # for demonstration, take 10 books
    print(book.barcode, book.primary_language, book.token_count)
    for paragraph in book.paragraphs:
        print(paragraph.text[:100])
        # do_something_with_paragraph(paragraph)

for book in islice(books, 10):
    for section in book.sections:
        # do_something_with_section(section)
        for paragraph in section.paragraphs:
            print(paragraph.text[:100])
        print()                    # add newline between sections

```

Books at the dataset level can be filtered by language or token count range. Specifically, `books.filter()` accepts optional `language`, `token_count_min`, or `token_count_max` arguments. Both token counts accept integer arguments and restrict to books with the specified token range; language accepts either one or a list of ISO 639-3 language strings and restricts to books whose primary language is one of those specified. For example, to iterate over English books with at least 1000 tokens:

```

for book in books.filter(language='eng', token_count_min=1000):
    # do_something_with_book(book)
    pass

```

For paragraph-level filtering: `books.paragraphs.filter()` accepts `language` (as a single string or a list, as above), `deduplicated` (Boolean: if true, exclude duplicate paragraphs from iteration), `bpb_min` and `bpb_max` (as floats). The per-volume BPB values are available as attributes for convenient reference (cf. Table 32). Section filtering uses the same semantics on `books.sections.filter()` and passes each filter to the paragraphs within each section. Sections where all paragraphs are filtered out are entirely excluded from iteration.